comput. complex. (2025) 34:18
© The Author(s) 2025
1016-3328/25/020001-53
published online October 29, 2025
https://doi.org/10.1007/s00037-025-00276-5

computational complexity

LIFTING DICHOTOMIES

Yaroslav Alekseev, Yuval Filmus,
and Alexander V. Smal

Abstract. Lifting theorems are used to transfer lower bounds between Boolean function complexity measures. Given a lower bound on a complexity measure A for some function f, we compose f with a carefully chosen *gadget* function g and get essentially the same lower bound on a complexity measure B for the *lifted* function $f \diamond g$. Lifting theorems have applications in many different areas, such as circuit complexity, communication complexity, proof complexity, etc.

One of the main questions in the context of lifting is how to choose a suitable gadget g. Generally, to get better results, i.e., to minimize the losses when transferring lower bounds, we need the gadget to be of a constant size (number of inputs). Unfortunately, in many settings we know lifting results only for gadgets of size that grows with the size of f, and it is unclear whether they can be improved to constant-size gadgets. This motivates us to identify the properties of gadgets that make lifting possible.

In this paper, we systematically study the question: 'For which gadgets does the lifting result hold?' in the following four settings: lifting from decision tree depth to decision tree size, lifting from conjunction DAG width to conjunction DAG size, lifting from decision tree depth to parity decision tree depth and size, and lifting from block sensitivity to deterministic and randomized communication complexities. In all the cases, we prove the complete classification of gadgets by exposing the properties of gadgets that make lifting results hold. The structure of the results shows that there are no intermediate cases—for every gadget, there is either a polynomial lifting or no lifting at all. As a byproduct of our studies, we prove the log-rank conjecture for the class of functions that can be represented as $f \diamond \mathrm{OR} \diamond \mathrm{XOR}$ for some function f.

Keywords. decision trees, log-rank conjecture, lifting, parity decision trees

Subject classification. 68Q11 Communication complexity, information complexity

1. Introduction

For $f\colon \{0,1\}^n \to V$ and $g\colon \{0,1\}^m \to \{0,1\}$, a *(block-)composition* $f \diamond g\colon \{0,1\}^{n\times m} \to V$ is defined by

$$(f \diamond g)(z_1, z_2, \ldots, z_n) := f(g(z_1), g(z_2), \ldots, g(z_n)),$$

where each $z_i \in \{0,1\}^m$. Conceptually, *lifting theorems* have the following general form:

$$B(f \diamond g) = \Omega(A(f)),$$

where A and B are two complexity measures. Usually inside $\Omega(\cdot)$, there is also a term for dependency on g. In this context, we call the function g a *gadget* and say that *there is a (linear) lifting from A to B*. The first example of a lifting theorem appeared in Raz & Mckenzie (1999), where Raz and McKenzie proved a separation for the hierarchy of monotone circuit complexity classes within NC using a query-to-communication lifting theorem.

The need for such theorems is due to the fact that, in many cases, communication complexity lower bounds are much more difficult to prove compared to query complexity lower bounds. Lifting theorems proved to be useful in many other scenarios: proving communication complexity separations (Chattopadhyay *et al.* 2019; Göös & Pitassi 2018; Göös *et al.* 2018; Gs *et al.* 2017), proof complexity separations (Göös *et al.* 2016; Göös & Pitassi 2018; Huynh & Nordström 2012; de Rezende *et al.* 2020), monotone circuit complexity separations (Garg *et al.* 2020; Raz & Mckenzie 1999), etc.

While the lifting technique is widely used in different areas, we still do not quite understand its limitations. In the query-to-communication lifting theorems, we want to lower bound the deterministic/randomized communication complexity of the lifted function by deterministic/randomized query complexity of the original function. These theorems usually have the following form:

$$\mathrm{D}(f \diamond g) = \mathrm{DT}(f) \cdot \Theta(\log n),$$

where the gadget g has at most logarithmic communication complexity. In all the lifting theorems of this type, the *size* (the number of inputs) of the gadget g grows with the number of inputs of f (e.g., in Lovett *et al.* (2022) the size of the gadget is $n^{1+\varepsilon}$, where n is a number of inputs of the lifted function). Even though some results do not depend on the gadget size (e.g., Göös *et al.* 2018), in many scenarios, the use of non-constant-size gadgets leads to weaker results in the applications (e.g., when lifting theorems are used to prove monotone circuit lower bounds via communication complexity). It is unknown whether it is possible to prove a query-to-communication lifting theorem with a constant-size gadget, but we tend to believe that such a lifting exists.

At the same time, for other complexity measures, there are lifting theorems that can accommodate constant-size gadgets. Here are some examples of such lifting theorems:

1. Lifting decision tree depth to decision tree size (XOR gadget (Urquhart 2011)).
2. Lifting decision tree depth to parity decision tree depth/size (stifling gadgets (Chattopadhyay *et al.* 2023); INDEX gadget (Beame & Koroth 2023)).
3. Lifting from critical block sensitivity to communication complexity (VER gadget (Göös & Pitassi 2018)).
4. Lifting parity decision tree depth to communication complexity (XOR gadget (Hatami *et al.* 2018)).
5. Lifting AND decision tree depth to communication complexity (AND gadget (Knop *et al.* 2021)).
6. Lifting block sensitivity to randomized communication complexity (AND gadget (Zhang 2009)).

Note that in examples 4 and 5, the communication complexity is lower bounded by *some power* of the parity decision tree complexity or the AND decision tree complexity (in the latter case, with an additional $\log n$ factor). In such cases, we say that there is a *polynomial lifting*.

As mentioned earlier, lifting theorems that lower bound communication complexity by a *linear* function of query complexity are only known for non-constant-size gadgets. But in the case of polynomial lifting, there are query-to-communication complexity lifting theorems with constant-size gadgets. E.g., in example 6 the author lifts block sensitivity to randomized communication complexity with a constant-size gadget. Given that block sensitivity is polynomially related to query complexity, this gives a polynomial lifting from query complexity to communication complexity.

All examples of lifting theorems with constant-size gadgets that we mentioned above use specific and simple gadgets. But what happens if we plug in some other gadget? The same proof might not work, but would the lifting result still be true? That is not always clear. This reflects our lack of understanding of the properties of the gadget that make lifting possible. For these theorems, it is natural to pose the following question:

For which gadgets does the lifting result hold?

A systematic study of this question will help us to better understand how lifting works and what the requirements are for the gadget. It would be especially interesting to understand for which gadgets lifting fails. This is what we need, for example, if we want to find a good candidate for query-to-communication linear lifting with a constant-size gadget.

One of the areas that might benefit from new lifting theorems is the study of *the log-rank conjecture* (Lovsz & Saks 1988). The log-rank conjecture states that the deterministic communication complexity of a function is polynomially related to the logarithm of the real rank of its associated communication matrix. Currently there is a exponential gap between the lower bound $\Omega\left(\log^2(\operatorname{rank}(f))\right)$ (Göös *et al.* 2018) and the upper bound $O\left(\sqrt{\operatorname{rank}(f)}\cdot\log\operatorname{rank}(f)\right)$ (Lovett 2016). It seems that the general case of this problem is out of reach now. So, most research in this area has concentrated on the study of various complexity measures and special cases, such as composed functions (e.g., see Knop *et al.* 2021). Therefore, lifting in this area is one of the main proof techniques.

The other area craving new lifting theorems is proof of complexity. There is a tight connection between proof complexity and

lifting theorems. For example, lifting decision tree depth to parity decision tree depth/size from Chattopadhyay *et al.* (2023) and Beame & Koroth (2023) provides a systematic way to prove tree-like Res($\oplus$) size lower bounds. It is important to mention that for proof complexity, we usually need to lift theorems that hold for relations.

Finally, we think that a large number of different applications make lifting a technique that is worth exploring on its own. Moreover, a deeper understanding of how lifting works and what it requires from gadgets can lead to new applications and results.

1.1. Our results and methods. In this paper, we systematically explore the question '*For which gadgets does the lifting result hold?*' in several different settings. We prove complete classifications of gadgets in the following four settings:

- lifting from decision tree depth to decision tree size,
- lifting from certificate complexity to conjunction DAG size,
- lifting from decision tree depth to parity decision tree depth and size,
- lifting from block sensitivity to deterministic and randomized communication complexities.

All these results are formulated as dichotomies (a trichotomy in one of the cases) that essentially state that there is either polynomial lifting (even with constant-size gadgets) or no lifting at all, with no intermediate cases. As a byproduct of our studies, we prove the log-rank conjecture for the class of functions that can be represented as $f \diamond \mathrm{OR} \diamond \mathrm{XOR}$ for some function f.

Now we describe the results in more detail and give an overview of the proof methods.

1.1.1. Decision tree depth to size. In Section 3, we define a class of *resistant* gadgets (defined in Section 3.1) and prove a decision tree depth to decision tree size lifting theorem for this class of gadgets (see Theorem 3.2). We give two different proofs illustrating the ideas of two different approaches: proof by simulation

and proof using random projections. The proofs later in the paper refer to the proofs in this simple case. The proof by simulation is constructiveit shows how, given a decision tree for the lifted function $f \diamond g$, one can construct a decision tree for the original function f, such that the depth of the new tree is bounded by a logarithm of the size of the given tree. In the proof using random projections, we use the probabilistic method to show that there is a projection that converts the decision tree for $f \diamond g$ into a shallow decision tree for f.

Our goal is to prove a classification theorem, so we need to find a class of gadgets such that lifting works only for gadgets in this class. It appears that the class of resistant gadgets is too small for this. We define a wider class of *weakly resistant* gadgets (defined in Section 3.2) and prove a certificate complexity to decision tree size lifting theorem (see Theorem 3.4). The proof uses the random projections method. Note that the decision tree size is upper bounded by the certificate complexity squared, so as a corollary we get a polynomial lifting from decision tree depth to decision tree size (see Corollary 3.5).

Finally, we give a complete classification of gadget functions in the context of polynomial lifting from decision tree depth to decision tree size. It is stated as a dichotomy result (see Theorem 3.9): either a gadget is weakly resistant and there is a polynomial lifting or there is no lifting at all.

In Section 3.4, we briefly discuss that some of the results above can be generalized to the case of search problems. Unfortunately, that does not give a classification theorem because the decision tree depth of a search problem can be exponentially greater than its certificate complexity. However, we state a conjecture that there is a decision tree depth to decision tree size polynomial lifting for weakly resistant gadgets.

1.1.2. Certificate complexity to conjunction DAG size. In Section 4, we generalize the results from the previous section to conjunction DAGs (defined in Section 4.1). First of all, in Section 4.2, we show that, similar to decision trees where depth and size are exponentially separated, there is an exponential separation between conjunction DAG width and size. The separation

is achieved for the Tribes function. In Section 4.3, we show that there is an exponential separation between decision tree size and conjunction DAG size in the case of relations. Indeed, conjunction DAGs can capture the structure of Resolution proofs, while decision trees can only capture the structure of tree-like Resolution proofs. Thus, the separation between these proof systems implies the separation between the measures under consideration. Finally, we argue that the lifting theorems from Section 3 can be generalized to the case of conjunction DAGs (see Theorem 4.1 and Theorem 4.2) using essentially the same proofs. Thus, we have a dichotomy result (see Theorem 4.3).

1.1.3. Decision tree depth to parity decision tree depth and size. Recently, Chattopadhyay *et al.* (2023) showed that if g is *stifling* (defined in Section 5.1), then $\log \mathrm{DTSize}_{\oplus}(f \diamond g) = \Theta(\mathrm{DT}(f))$. In Section 5, we extend their result providing the complete classification of the gadgets for decision tree depth to parity decision tree depth and size lifting.

In Section 5.2, we state and prove a 'minimum weight lemma' (see Lemma 5.3), the technical lemma that we will use several times. This lemma states that if the certificate complexity of some function f is large enough in comparison to the parity certificate complexity of the lifted function $f \diamond g$ at some input, then there is a substitution such that the minimal parity certificate of $f \diamond g$ at this input has a large Hamming weight. The notion of minimum weight turns out to be surprisingly powerful in the context of lifting. For example, Lemma 5.8 shows that if $\mathrm{C}_{\oplus}$ is a parity certificate for $f \diamond \mathrm{OR}_2$ with large minimum weight, then it contains a point z such that $\mathrm{OR}_2^n(z) = (1, \ldots, 1)$, which provides heavy restrictions on the parity certificates of $f \diamond \mathrm{OR}_2$ that certify the opposite value.

In Section 5.3, we consider the most challenging case of this liftingthe case of OR gadget. In Theorem 5.5, we show that there is at least a quadratic gap in the certificate to parity certificate complexity lifting for OR gadget. If we assume that the lifting result in Chattopadhyay *et al.* (2023) holds for OR gadget as well then the quadratic separation is tight (see Proposition 5.6). In Section 5.3.2, we show a polynomial (cubic) lifting for OR gadget (see

Theorem 5.7). The proof consists of three ingredients. First, we show that if the minimum weight of a parity certificate for the lifted function is at least the size of the parity certificate, then this certificate covers the all-1 input for the inner function (see Lemma 5.8). Then we use it to upper bound the minimum weight of a parity certificate in terms of the sizes of parity certificates for 0 and 1 (see Lemma 5.9). And finally, we apply the 'minimum weight lemma' (see Lemma 5.10). In the proof of Theorem 5.7, we assume that the certificate complexity of the original function is much larger than the parity certificate complexity of the lifted function, but due to the 'minimum weight lemma,' it would contradict the upper bound on the minimum weight of the parity certificate.

In Section 5.4, we prove a lifting from certificate complexity to parity decision tree size for AND/OR gadgets, the gadgets that affine project to both binary AND and binary OR (see Theorem 5.12). The proof is by simulation, similar to the simulation proof in Chattopadhyay *et al.* (2023).

Finally, in Section 5.5, we prove the trichotomy result that gives us a complete classification of gadgets (see Theorem 5.15). The classification is based on the fact that if a gadget is not AND/OR then it is a disjunction or a conjunction of affine forms (see Lemma 5.14). We show that only three cases are possible:

1. there are simultaneously a lifting from decision tree depth to parity decision tree depth and a lifting from certificate complexity to parity certificate complexity (AND/OR gadgets);

2. there is a lifting from decision tree depth to parity decision tree depth but no lifting between certificate complexities (gadgets that affine project to OR);

3. there is no lifting (all other gadgets, constant or affine).

In Section 5.6, we show an alternative proof for the lifting from decision tree depth to parity decision tree depth using function degree and sparsity (see Theorem 5.16). In the proof, we show that the degree of a function is upper bounded by the logarithm of the sparsity of the function lifted with OR gadget and compose it with a number of previously known inequalities. As a byproduct,

we get a proof of the log-rank conjecture for the class of functions that can be represented as $f \diamond \mathrm{OR} \diamond \mathrm{XOR}$ (see Theorem 5.17).

1.1.4. Block sensitivity to communication complexity. In Section 6, we provide a complete classification of gadgets for lifting from block sensitivity to deterministic and randomized communication complexities. Note that this classification also gives us a classification of gadgets for query-to-communication polynomial lifting since block sensitivity and query complexity are polynomially related to each other in the case of total functions. However, since the proof goes through block sensitivity, we classify block sensitivity to communication complexity lifting.

We start with the lifting theorem of Zhang (2009) (see Theorem 6.1) that works for gadget g iff both AND and OR reduce to g via a communication complexity reduction (defined in Section 6.1). Then we show that if there is no reduction from OR to a gadget g then the communication matrix of g is (up to rearrangement) block diagonal (see Lemma 6.3). This gives us a classification of gadgets in the context of reductions from OR, AND, and XOR (see Corollary 6.5 and Corollary 6.6).

In Section 6.2, we prove a dichotomy result for randomized communication complexity (see Theorem 6.7). The proof is based on the fact that if OR does not reduce to a gadget g, then $\mathrm{AND}_n \diamond g$ is essentially an instance of the equality function, and hence, its randomized communication complexity is logarithmic. If AND does not reduce to a gadget, the situation is symmetrical (see Lemma 6.8). Thus, Zhang's theorem covers all gadgets for which there is a lifting.

In Section 6.3, we prove the dichotomy result for deterministic communication complexity (see Theorem 6.10). The proof of the dichotomy depends on a block sensitivity to deterministic communication complexity lifting theorem that works for gadget g iff both AND and XOR reduce to g (see Theorem 6.9). Together with Zhang's lifting theorem that gives us a complete classification of gadgets: if at least two of three functions AND, OR, XOR reduce to gadget g, then there is a block sensitivity to deterministic communication complexity polynomial lifting. Otherwise, if at least two functions from this list do not reduce to g, then g is ei-

ther a constant function or (essentially) one of the functions AND, OR, XOR. The proof of the lifting theorem requires a number of ingredients. First of all, we compose two results from Hatami *et al.* (2018) and from Zhang & Shi (2010) to get a parity certificate to deterministic communication complexity polynomial lifting for XOR gadget (see Corollary 6.11). Then we show two lower bounds for specific cases: we show that the deterministic communication complexity of $f \diamond \mathrm{XOR}$ is lower bounded by the size of any minimal sensitive block of the function f (see Lemma 6.12), and that the deterministic communication complexity of $f \diamond \mathrm{AND}$ is lower bounded by the block sensitivity of f at the all-0 input (see Lemma 6.13). One of the ingredients is again the 'minimum weight lemma' (see Lemma 6.14). And the last one is the lemma, which shows that parity certificates of high minimum weight always intersect with (regular) certificates of sufficiently small size (see Lemma 6.15). Putting all the ingredients together, we get the desired lifting result.

2. Prerequisites

2.1. Notation. All the logarithms are base 2. We use OR_n, AND_n, XOR_n to denote, respectively, logical 'and,' 'or,' and 'exclusive or' of n Boolean inputs. For simplicity, we also define $\mathrm{OR} := \mathrm{OR}_2$, $\mathrm{AND} := \mathrm{AND}_2$, and $\mathrm{XOR} := \mathrm{XOR}_2$. We use the notation x^B for $x \in \{0,1\}^n$ and $B \subseteq [n]$ to denote x with all the coordinates in B flipped, i.e., $x_i = x_i^B \iff i \notin B$. For $g\colon \{0,1\}^k \to \{0,1\}$, we define[1] a function $g^n\colon \{0,1\}^{n\times k} \to \{0,1\}^n$ such that

$$g^n(x_1, x_2, \ldots, x_n) := (g(x_1), g(x_2), \ldots, g(x_n)).$$

For a function $f\colon \{0,1\}^n \to \{0,1\}$, an input $x \in \{0,1\}^n$, and a partial assignment $\sigma \in \{0,1,*\}^n$ that agrees with x on all non-$*$ coordinates, we use $f|_\sigma$ to denote a restriction of f to σ and $x|_\sigma$ to denote a projection of x to the $*$-coordinates of σ. For

[1] In some papers, lifting theorems are formulated for the classical composition operation '$\circ$' rather than for the block composition '$\diamond$.' Note that for $f\colon \{0,1\}^n \to \{0,1\}$ and $g\colon \{0,1\}^k \to \{0,1\}$, $f \diamond g \equiv f \circ g^n$.

a gadget $g\colon \{0,1\}^k \to \{0,1\}$ and a *blockwise* partial assignment $\sigma \in \left(\{0,1\}^k \cup \{*^k\}\right)^n$ (i.e., in any block of variables that corresponds to one copy of g, the variables are either all set or all stars), we use $g^n(\sigma)$ to denote the partial assignment in $\{0,1,*\}^n$ induced by applying g to non-$*$ blocks of σ. We use '$\sqcup$' instead of '$\cup$' to indicate a union of disjoint sets.

2.2. Complexity measures. Throughout the text, we will consider several complexity measures.

Decision tree complexity A *decision tree* is a rooted binary tree with internal nodes labeled by input variables (representing queries), edges labeled with 0 and 1, and leaves labeled by some values. Decision tree evaluation is defined in a natural way: given an assignment for the input variables, we traverse the tree starting from the root and then sequentially choose every next edge according to the given assignments until we reach some leaf. The label of this leaf is the result of the evaluation. A decision tree computes some function f if for all possible assignments to the input variables, the decision tree evaluates to the value of the function. For a function f, $\mathrm{DT}(f)$ denotes the minimal depth of a decision tree computing f, and $\mathrm{DTSize}(f)$ denotes the minimal number of leaves in a decision tree computing f.

Parity decision tree complexity A *parity decision tree* extends the concept of a decision tree by allowing queries that are arbitrary linear combinations of the input variables. It can be described as a rooted binary tree with internal nodes labeled by linear combinations of the input variables, edges labeled with 0 and 1, and leaves labeled by some values. The evaluation is defined analogously. For a function f, $\mathrm{DT}_{\oplus}(f)$ denotes the minimal depth of a parity decision tree computing f, and $\mathrm{DTSize}_{\oplus}(f)$ denotes the minimal number of leaves in a parity decision tree computing f.

Certificate complexity A *certificate complexity* is a non-deterministic analogue of the decision tree complexity. A *certificate* for an input $x \in \{0,1\}^n$ to a function f is a set $S \subseteq [n]$ of indices

such that f restricted to all inputs that match x on S is constant, i.e., $f(y) = f(x)$ whenever $y|_S = x|_S$. The certificate complexity $\mathrm{C}(f, x)$ of f at input x is the size of the smallest certificate for x. Finally, the certificate complexity of function f is defined as $\mathrm{C}(f) := \max_{x \in \{0,1\}^n} \mathrm{C}(f, x)$.

Parity certificate complexity A *parity certificate* for an input $x \in \{0,1\}^n$ to a function f is a set A of linear forms such that f is constant on the affine subspace defined by $A = A(x)$. A *parity certificate complexity* $\mathrm{C}_\oplus(f, x)$ of a function f at input x is the size (number of linear forms) of the smallest parity certificate for x. The parity certificate complexity of f is defined as $\mathrm{C}_\oplus(f) := \max_{x \in \{0,1\}^n} \mathrm{C}_\oplus(f, x)$.

Block sensitivity A block $B \subseteq [n]$ is *sensitive* for a function f at x iff $f(x) \neq f(x^B)$. The *block sensitivity* $\mathrm{bs}(f, x)$ of f at x is the maximum number of disjoint sensitive blocks for f at x. The block sensitivity $\mathrm{bs}(f)$ of f is the maximum sensitivity $\mathrm{bs}(f, x)$ over all points $x \in \{0,1\}^n$.

Degree and sparsity of the function Every Boolean function defined on $\{0,1\}^n \to \{0,1\}$ can be considered as a function $f\colon \{-1,1\}^n \to \{-1,1\}$ by substituting 0 and 1 with 1 and -1, respectively. Every such function has a unique representation as a multilinear polynomial over $\mathbb{R}$ of the following form

$$f(x) = \sum_{S \subseteq [n]} c_S x_S, \qquad \text{where } x_S = \prod_{i \in S} x_i.$$

The *degree* of f is defined as the degree of the corresponding polynomial, and we denote it by $\deg(f)$. The *sparsity* of f is the number of nonzero coefficients in this representation, and we denote it by $\mathrm{spar}(f)$.

Deterministic communication complexity and rank For a function $f\colon X \times Y \to Z$, consider the following game for two players, Alice and Bob, that want to compute f. Alice and Bob

are given $x \in X$ and $y \in Y$, respectively. Their goal is to compute $f(x, y)$. In order to do it, the players exchange information about their parts of the input using a simple communication channel that allows sending bit messages. Before the game, the players come up with a *communication protocol* that determines their behavior on all possible inputs. The cost of a protocol is the maximum total number of bits sent by the players over all possible inputs $x \in X$, $y \in Y$. The *deterministic communication complexity* of f is the minimal cost of a deterministic communication protocol that computes f. We denote it by $\mathrm{D}(f)$.

For a function $f\colon X \times Y \to \{0,1\}$, a *communication matrix* of f is a matrix $M_f \in \{0,1\}^{X \times Y}$ such that $(M_f)_{x,y} := f(x,y)$ for all $x \in X$, $y \in Y$. We define $\mathrm{rank}(f)$ to be the rank of the matrix M_f over the reals.

Randomized communication complexity In a *randomized communication game*, Alice and Bob have access to an unlimited amount of random bits, and they can use it to decide which bit to send next. We say that a *(private coin) randomized communication protocol* Π computes a function $f\colon X \times Y \to Z$ with error ε if

$$\Pr_{r_A, r_B}\big[\Pi(x, y, r_A, r_B) = f(x,y)\big] \geq 1 - \varepsilon, \quad \forall x \in X,\, y \in Y,$$

where r_A and r_B are the strings of random bits used by Alice and Bob, respectively. The cost of a randomized protocol is the maximum number of bits that can be sent. We denote by $\mathrm{R}_\varepsilon(f)$ the minimal cost of a private coin randomized communication protocol that computes f with error ε. The *randomized communication complexity* of f is defined as $\mathrm{R}(f) := \mathrm{R}_{1/3}(f)$.

3. Decision tree depth to size

We start by exploring perhaps one of the simplest scenarios, depth-to-size lifting in decision trees. To give a flavor of the techniques that we are going to use in this paper, we prove a lifting theorem for the class of *resistant* gadgets.

3.1. Resistant gadgets. Urquhart (2011) proved that for any function $f\colon \{0,1\}^n \to \{0,1\}$,

$$\log \mathrm{DTSize}(f \diamond \mathrm{XOR}) = \Omega(\mathrm{DT}(f)).$$

We generalize this result for the class of *resistant* gadgets.

Definition 3.1. *A gadget $g\colon \{0,1\}^m \to \{0,1\}$ is* resistant *if for every $i \in [m]$ and $b \in \{0,1\}$, the function obtained by fixing the ith input to b is not constant. Equivalently, the minimum certificate complexity at inputs is larger than 1. E.g., the* XOR *function is resistant while the* AND *function is not.*

Theorem 3.2. *For any function $f\colon \{0,1\}^n \to \{0,1\}$ and a resistant gadget $g\colon \{0,1\}^m \to \{0,1\}$,*

$$\log \mathrm{DTSize}(f \diamond g) = \Omega(\mathrm{DT}(f)).$$

We give two proofs of this theorem using two different approaches: *simulation* and *random projections*. The first proof is clearer and, in some sense, constructive, but the second proof can be generalized to other settings. In the following proofs, we use $x_1, \ldots, x_n$ to refer to the inputs of function f. The inputs to the lifted function $f \diamond g$ are denoted by $z_{i,j}$, where $i \in [n]$ and $j \in [m]$.

Proof (Proof by simulation). Given a decision tree T for $f \diamond g$ of size S, we construct a decision tree for f of depth at most $\log S$. We traverse T from the root to a leaf and maintain a partial assignment to z which is consistent with the input x, in the sense that the partial assignment can be extended to an input z which satisfies $x = g^n(z)$ (i.e., $x_i = g(z_{i,1}, \ldots, z_{i,m})$). This ensures that if the leaf is labeled b, then $f(x) = b$.

Initially, the partial assignment is empty. When at a node labeled $z_{i,j}$, there are two options. If the value of $z_{i,j}$ is already known, then we follow the edge labeled with its value. Otherwise, we set $z_{i,j}$ to the value which corresponds to the smaller subtree, query x_i, and set the remaining $z_{i,k}$'s so that $x_i = g(z_{i,1}, \ldots, z_{i,m})$ (this is possible since g is resistant).

Each time we make a query to x, the size of the current tree is at least halved (the size of the smaller child subtree of T' is at most half the size of T'), and hence the process terminates after at most $\log S$ queries to x. □

Proof (Proof using random projections). Let T be a decision tree for $f \diamond g$ of size S. We apply the following random projection. For each $i \in [n]$, choose an index $j \in [m]$ at random, and set $z_{i,j}$ to a random value $b \in \{0,1\}$. Set the remaining variables $z_{i,k}$, $k \neq j$, to $x_i, \bar{x}_i, 0, 1$ in such a way that $g(z_{i,1}, \ldots, z_{i,m}) = x_i$. This is possible since g is resistant, as we show below. The result is a decision tree T' for f.

To see that we can always construct such a projection, let $h(z)$ be a non-constant function, and choose two inputs w^0, w^1 such that $h(w^b) = b$. We define the projection ρ as follows: if $w_i^0 = w_i^1 = a$ then set $z_i = a$; otherwise, if $w_i^0 = 0$ then set $z_i = x$, and if $w_i^1 = 0$ then set $z_i = \bar{x}$. By construction, $\rho|_{x=b} = w^b$ and so $h|_\rho(b) = h(w^b) = b$.

For example, suppose that $h(z_1, z_2, z_3)$ is the majority function on three inputs and that we set the first bit z_1 to 0. Then $h(0, 1, x_i) = x_i$ and $h(0, x_i, x_i) = x_i$ are two possible ways to choose a pair of inputs (w^0, w^1) corresponding to $(010, 011)$ and $(000, 011)$, respectively.

We would like to say that there is a choice of random projection for which T' is shallow. To do this, let us calculate the probability that some vertex v at depth D in T survives in T'. The path leading to v queries variables from at least D/m different blocks. For each such block, choose an arbitrary variable that is queried along the path and consider the value the path assigns to it. If the random projection sets this variable to the opposite value then the vertex v does not survive in T'. It happens with probability at least $1/(2m)$ for each variable, and so the vertex v survives with probability $p \leq (1 - 1/(2m))^{D/m} = 1/2^{O(D)}$.

If $pS < 1$, then the expected number of vertices at depth D surviving in T' is less than 1, and so there is a random projection for which T' has a depth less than D. This happens for some $D = O(\log S)$. □

3.2. Weakly resistant gadgets. There are gadgets, such as $x \vee (y \wedge z)$, which are clearly not resistant, but for which the lifting still holds as we will show below. To capture these cases, we define a more general class of *weakly resistant* gadgets.

DEFINITION 3.3. *A gadget $g\colon \{0,1\}^m \to \{0,1\}$ is* weakly resistant *if for every certificate α (a partial assignment which sets the value of g) there is a substitution of one input variable (with 0 or 1) which conflicts with α and does not make g constant.*

Every resistant gadget is trivially weakly resistant: we can take any variable mentioned in α and substitute the opposite value. The aforementioned gadget $h(x,y,z) = x \vee (y \wedge z)$ is weakly resistant (as we show below) but not resistant, since $h|_{x=1} \equiv 1$. The gadget $x \vee y$ is not even weakly resistant due to the certificate $x = y = 0$: if we substitute $x = 1$ or $y = 1$ then the gadget becomes constant.

Let us verify that h is weakly resistant by considering all possible certificates:

- $x = 1$: take $x = 0$.
- $y = z = 1$: take $y = 0$ or $z = 0$.
- $x = y = 0$: take $y = 1$.
- $x = z = 0$: take $z = 1$.

We now prove the following lifting theorem:

THEOREM 3.4. *For any $f\colon \{0,1\}^n \to \{0,1\}$ and a weakly resistant gadget $g\colon \{0,1\}^m \to \{0,1\}$ the following holds:*

$$\log \mathrm{DTSize}(f \diamond g) = \Omega(\mathrm{C}(f)).$$

PROOF. Given a decision tree T for $f \diamond g$ of size S, we construct a blockwise random projection ρ in the following way. For each $i \in [n]$, we choose a random certificate C_i of g. Let $z_{i,j} = b$ be the substitution promised by weak resistance for C_i that does not make $g(z_i)$ constant. We can complete $\rho|_{z_i}$ to a projection such that $g(\rho|_{z_i}) = x_i$ (as in the case of resistant predicates, see proof of Theorem 3.2 using random projections).

Let ℓ be a leaf of T and τ be the corresponding partial assignment to the variables of $f \diamond g$. Suppose that τ contains certificates for at least D blocks (i.e., in these blocks, the value of g on τ is fixed). If there are M certificates of g, then τ is consistent with ρ with probability at most $(1 - 1/M)^D$. Choose $D = \Theta(\log S)$ so that $(1 - 1/M)^D S < 1$. By union bound, under some choice of ρ, the partial assignments for all leaves of $T|_\rho$ contain certificates for less than D blocks. Since by construction $(f \diamond g)|_\rho = f$, this shows that $\mathrm{C}(f) < D$.

Indeed, given an input x to f, consider the corresponding input $\rho(x)$ to $f \diamond g$. This input reaches some leaf ℓ, in which less than D blocks contain certificates, say the blocks in $I \subseteq [n]$. For each input x' consistent with gadget values for blocks in I, we can find an input z reaching the same leaf such that $g^n(z) = x'$. Hence, the block certificates in ℓ correspond to a certificate of f. □

Remark. *One of the anonymous reviewers noticed that the structure of a decision tree is completely irrelevant to the above proof. We only use the fact that the leaves are monochromatic and cover all the inputs. Thus, Theorem 3.4 can be restated by replacing the decision tree size with the subcube cover number. This also explains why, in the next section, we will be able to reuse the above proof in a similar lifting result for conjunction DAGs (see Theorem 4.2).*

Corollary 3.5. *For any $f\colon \{0,1\}^n \to \{0,1\}$ and a weakly resistant gadget $g\colon \{0,1\}^m \to \{0,1\}$ the following holds:*

$$\log \mathrm{DTSize}(f \diamond g) = \Omega\big(\sqrt{\mathrm{DT}(f)}\big).$$

Proof. Beals *et al.* (2001) shows that for any function f,

$$\mathrm{DT}(f) \leq (\mathrm{C}(f))^2.$$

Together with Theorem 3.4, that gives us the desired bound. □

NOTE 3.6. *This proof can also be phrased in the language of simulations. This was worked out jointly with Amit Chakrabarty, Mika Gs, Johan Hstad, Susanna de Rezende, Robert Robere, and Avishay Tal in a Dagstuhl breakout session.*

QUESTION 3.7. *We do not know whether the lower bound is tight, even for* $h = x \vee (y \wedge z)$. *At the moment we can't even rule out* $\log \mathrm{DTSize}(f \diamond h) = \Omega(\mathrm{DT}(f))$.

Sherstov (Sherstov 2010, Theorem 6.4) proved that

$$\max\big(\log \mathrm{rank}(f \diamond \mathrm{AND}), \log \mathrm{rank}(f \diamond \mathrm{OR})\big) \geq \deg(f).$$

Since rank lower bounds decision tree size, this implies that

$$\log \mathrm{DTSize}(f \diamond g) \geq \deg(f).$$

Note that we are not aware of any separation between $\mathrm{DT}(f)$ *and* $\max(\mathrm{C}(f), \deg(f))$.

3.3. Gadget classification. Now, we can prove a dichotomy result. First of all, we need the following classification of gadgets.

LEMMA 3.8. *For every* $g\colon \{0,1\}^m \to \{0,1\}$, *one of the following cases holds:*

(i) Function g *is a (possibly empty) conjunction or a disjunction of literals.*

(ii) Function g *is weakly resistant.*

PROOF. Let $y_1, \ldots, y_m$ denote the inputs of g. If g is not weakly resistant then there is a certificate α such that for every substitution $y_j = b$ conflicting with α, the function $g|_{y_j=b}$ is constant. Moreover, we can assume that α is an inclusion-minimal certificate.

Suppose that $\alpha = \{y_{i_1} = b_1, \ldots, y_{i_t} = b_t\}$ and $g|_\alpha = b$. By assumption, the function $g|_{y_{i_s}=\bar{b}_s}$ is constant. Since α is inclusion-minimal, necessarily $g|_{y_{i_s}=\bar{b}_s} = \bar{b}$. Thus if $b = 1$ then

$$g = [y_{i_1} = b_1] \wedge \cdots \wedge [y_{i_t} = b_t],$$

otherwise, if $b = 0$ then

$$g = [y_{i_1} \neq b_1] \vee \cdots \vee [y_{i_t} \neq b_t]. \qquad \square$$

We can now state the dichotomy theorem.

THEOREM 3.9. *For every gadget* $g\colon \{0,1\}^m \to \{0,1\}$, *one of the following cases holds:*

(i) There is an infinite family of functions f_n *with* $\mathrm{DT}(f_n) \to \infty$ *and* $\mathrm{DTSize}(f_n \diamond g) = O(\mathrm{DT}(f_n))$.

(ii) For every function f, $\log \mathrm{DTSize}(f \diamond g) = \Omega\left(\mathrm{DT}(f)^{\Omega(1)}\right)$.

PROOF. These two cases correspond to the cases of Lemma 3.8. If g is a disjunction then we take $f_n := \mathrm{OR}_n$. Clearly $\mathrm{DT}(f_n) = n$ and $\mathrm{DTSize}(f \diamond g) = O(n)$ (if g is constant, then $\mathrm{DTSize}(f_n) = 1$). The case of a conjunction is similar.

Otherwise, g is weakly resistant, thus by Corollary 3.5 we have $\log \mathrm{DTSize}(f \diamond g) = \Omega\left(\sqrt{\mathrm{DT}(f)}\right)$. $\square$

3.4. Generalization to search problems. Let f be a *relation*, $f \subseteq \{0,1\}^n \times V$. For a gadget $g\colon \{0,1\}^m \to \{0,1\}$, we define a composition $f \diamond g \subseteq (\{0,1\}^m)^n \times V$ as a relation, such that

$$(z_1, z_2, \ldots, z_n, r) \in (f \diamond g) \iff (g(z_1), g(z_2), \ldots, g(z_n), r) \in f.$$

One can observe that the proofs of Theorem 3.2 and Theorem 3.4 work as well when we let f to be a relation. However, the Corollary 3.5 does not hold for the relations since $\mathrm{DT}(f)$ can be exponentially greater than $\mathrm{C}(f)$. As an example of this, one can take f to be a *falsified clause problem* corresponding to the Pigeonhole Principle Formula over an expander graph. Ben-Sasson & Wigderson (2001) showed that *resolution width* (which is at most $\mathrm{DT}(f)$) of any refutation for this formula is at least $\Omega(n)$, but the certificate complexity is constant (since each of the clauses of the formula is constant-size).

However, we conjecture that lifting from decision tree depth to decision tree size can also be proved for weakly resistant gadgets. Equivalently, this will mean that such lifting holds for the gadget $g(x, y, z) := x \vee (y \wedge z)$.

4. Certificate complexity to conjunction DAG size

4.1. Conjunction DAGs. A conjunction DAG was first defined formally in Garg *et al.* (2020), though it appears implicitly in previous work. *A conjunction DAG* over a set of variables is a single-rooted DAG with the following additional information:

- Each internal vertex is annotated with a variable, and it has two outgoing edges, one labeled 0 and the other labeled 1.
- Each vertex v is annotated with a partial assignment $\rho(v)$ with the following constraint. Suppose that v queries x_i, and the answer b leads to the vertex v_b. Then the partial assignment $\rho(v_b)$ is a subset of the partial assignment $\rho(v) \cup \{x_i \leftarrow b\}$. (This is not identical to the definition in Garg *et al.* (2020), but morally the same.)
- The partial assignment at the root is the empty assignment.
- Each leaf is annotated with some value.

A decision DAG *computes* f if for every leaf ℓ annotated with y_ℓ, $\rho(\ell)$ is a y_ℓ-certificate of f.

Every decision tree is a decision DAG. There are three parameters of interest: the (total) size (number of vertices), the leaf size (number of leaves), and the width (maximum number of variables in any $\rho(v)$).

4.2. Size vs width. A decision tree of depth d contains at most 2^d leaves, and this is tight for the parity function, in the sense that the bound $\mathrm{DTSize}(f) \leq 2^{\mathrm{DT}(f)}$ cannot be improved when f is the parity function.

Similarly, a conjunction DAG of width d has at most $\binom{n}{\leq d} = O(n^d)$ vertices. This is almost tight for the following function (also known as the *Tribes* function)

$$f(x) = \bigvee_{i=1}^{\sqrt{n}} \bigwedge_{j=1}^{\sqrt{n}} x_{ij}.$$

We can construct a conjunction DAG of width $O(\sqrt{n})$ for f as follows. We think of the input as a matrix, where i is the row number and j is the column number. We scan each row sequentially. If the current row consists only of 1s, we stop. Otherwise, we add the first 0 to ρ, and forget all the remaining entries of the row.

Conversely, consider the set of $\sqrt{n}^{\sqrt{n}}$ inputs having a single 0 per row. No two inputs share the same certificate, and so every conjunction DAG for f must contain at least $n^{\sqrt{n}/2}$ leaves.

4.3. Separation between decision tree size and conjunction DAG size. Let $\mathrm{d}(f)$ denote the conjunction DAG width of a function f, which is the smallest width of a conjunction DAG for f. Since $\mathrm{DT}(f) \geq \mathrm{d}(f) \geq \mathrm{C}(f) = \Omega\big(\sqrt{\mathrm{DT}(f)}\big)$, in the case of functions conjunction DAG width and decision tree depth are polynomially related. In this section, we show that the size of the decision tree and the conjunction DAG can be far apart in the case of relations.

Alekhnovich *et al.* (2007) constructed a family of CNF contradictions ϕ_n with $\mathrm{poly}(n)$ many variables and clauses which have a refutation in Resolution of size $\mathrm{poly}(n)$, but such that any refutation in tree-like Resolution (even in regular Resolution) is of size $2^{\Omega(n)}$.

We can think of a Resolution proof as a conjunction DAG which solves the *falsified clause problem*: given a truth assignment, find a falsified clause. This is a relation rather than a function. Similarly, a tree-like Resolution proof is a decision tree solving the same problem.

Now we are going to show that there is a function $f\colon \{0,1\}^n \to \{0,1\}$ that can be computed by a conjunction DAG of size $\mathrm{poly}(n)$ such that $\mathrm{DTSize}(f) = 2^{\Omega(n/\log n)}$. Consider the contradictions ϕ_n mentioned above. Index the clauses of ϕ_n using bitstrings of length $\ell = O(\log n)$ in some arbitrary way. Now consider a polynomial size Resolution proof of ϕ_n, and let f_i be the function mapping a truth assignment to the ith bit of the bitstring indexing the falsified clause found by the proof. By construction, each f_i has a conjunction DAG of size $\mathrm{poly}(n)$. Given decision trees for $f_1, \ldots, f_\ell$, we can construct a decision tree solving the falsified clause problem

of size $\prod_i \mathrm{DTSize}(f_i)$. Since this product must be at least $2^{\Omega(n)}$, we conclude that $\max_i \mathrm{DTSize}(f_i) = 2^{\Omega(n/\log n)}$.

4.4. Gadget classification. For the case of conjunction DAGs, we can generalize Theorem 3.2:

THEOREM 4.1. *Let $f \subseteq \{0,1\} \times V$ be a relation and $g\colon \{0,1\}^m \to \{0,1\}$ be a resistant gadget. If there is a conjunction DAG of size S computing $f \diamond g$, then there is a conjunction DAG of width $O(\log S)$ computing f.*

PROOF. We argue that the random projection proof of Theorem 3.2 can be adapted for this theorem as well. The difference is that instead of killing terms of high depth, we now focus on the nodes with high width and show that with high probability, after random projection, there will be no such nodes in the DAG. All the calculations remain unchanged. □

Moreover, for conjunction DAGs we can prove an analog of Theorem 3.4:

THEOREM 4.2. *Let $f \subseteq \{0,1\} \times V$ be a relation and $g\colon \{0,1\}^m \to \{0,1\}$ be a weakly resistant gadget. If there is a conjunction DAG for $f \diamond g$ with S leaves, then the certificate complexity of f is at most $O(\log S)$.*

PROOF. The proof of Theorem 3.4 can be applied in this setting as is because it ignores the internal structure of the decision tree and considers only the leaves. □

Note that Theorem 4.2 gives us a lifting from certificate complexity to leaf size. Unfortunately, in the case of the falsified clause problem for CNF, this theorem cannot be effectively used to prove lower bounds since leaf size, as well as certificate complexity, is usually small. However, this theorem still implies a dichotomy result similar to one in Theorem 3.9.

THEOREM 4.3. *For every gadget $g\colon \{0,1\}^m \to \{0,1\}$, one of the following cases holds:*

(i) There is an infinite family of functions f_n such that $f_n \diamond g$ can be computed by a conjunction DAG having $O(n)$ leaves, but $\mathrm{C}(f_n) = \Omega(n)$.

(ii) For every relation f, if we have a conjunction DAG for $f \diamond g$ with S leaves, then the certificate complexity of f is at most $O(\log S)$.

Proof. The two cases of the theorem correspond to the two cases of Lemma 3.8. If g is a disjunction then we take $f_n := \mathrm{OR}_n$. Clearly, $\mathrm{C}(f_n) = n$ and $f_n \diamond g$ can be computed with a conjunction DAG (actually a tree) having $O(n)$ leaves (if g is constant, then the DAG has only one leaf). The case of a conjunction is similar.

Otherwise, gadget g is weakly resistant and by Theorem 4.2 we have $\log S = \Omega\big(\mathrm{C}(f)\big)$. □

5. Decision tree depth to parity decision tree depth and size

In this section, we are going to classify gadgets in the context of decision tree depth to parity decision tree depth and size lifting. We start by restating a recent result of Chattopadhyay *et al.* (2023). After that, we are going to state the 'minimum weight lemma' (Lemma 5.3), which is a technical tool we will use multiple times throughout this section and the following one. We use this lemma to prove the lifting from certificate complexity to parity certificate complexity with the OR gadget, which is the most challenging case in the classification. Finally, we will show an alternative and much simpler proof for the lifting from decision tree depth to parity decision tree depth using degree and sparsity. In some sense, this proof should give us a better exponent for the decision tree lifting. The main point of considering certificate complexity lifting is that there is a nontrivial upper bound for OR gadget, showing that it is impossible to prove a linear lifting in this setting (see Theorem 5.5). As a byproduct, we get a proof of the log-rank conjecture for the class of functions that can be represented as $f \diamond \mathrm{OR} \diamond \mathrm{XOR}$.

5.1. Stifling gadgets. Chattopadhyay *et al.* (2023) defined the following notion.

Definition 5.1. *A function $g\colon \{0,1\}^m \to \{0,1\}$ is* k-stifling *if for every set of k coordinates and $b \in \{0,1\}$ there is a way to set the remaining $m-k$ coordinates so that the output is b (regardless of the value of the chosen k coordinates). A function is* stifling *if it is* 1*-stifling.*

Chattopadhyay *et al.* (2023) showed that if g is stifling then $\log \mathrm{DTSize}_{\oplus}(f \diamond g) = \Theta(\mathrm{DT}(f))$, where the hidden constant can depend on g. See Appendix A for an exposition of their proof.

5.2. Minimum weight lemma. A parity certificate C is a system of affine equations over input variables. It can be represented by a matrix A and a vector b that define a linear subspace, i.e., $Ax = b$.

Definition 5.2. *The* minimum weight *of a parity certificate C is the minimum Hamming weight of nonzero vectors in the row space of A.*

For example, one can consider a certificate $\{x+y=1, x+y+z=0\}$. The minimum weight of this certificate is equal to 1, and this corresponds to the equation $z=1$. On the other hand, the minimum weight of a certificate $\{x+y=1, x+z=0\}$ is equal to 2.

The following lemma is a key tool that we will use both in Section 5 and in Section 6. This lemma shows that if the certificate complexity of some function f is large enough in comparison to the parity certificate complexity of the lifted function $f \diamond g$ at some input, then there is a substitution such that the minimal parity certificate of $f \diamond g$ at this input has large Hamming weight. Informally, this means that we can make a small enough substitution, such that the preimage of the parity certificate after the substitution will contain the points that we are interested in. For example, in the case of $g = \mathrm{OR}_2$, we will be able to show that if a parity certificate has a large weight, then it should contain a point z such that $\mathrm{OR}^n(z) = (1,\dots,1)$.

LEMMA 5.3 (minimal weight lemma). *For $f\colon \{0,1\}^n \to \{0,1\}$ and $g\colon \{0,1\}^m \to \{0,1\}$, suppose that $f \diamond g$ has parity certificate complexity at most k at some point x. Let K be a parameter. If the certificate complexity of f at $g^n(x)$ is greater than $k \cdot K$ then we can find a blockwise partial assignment σ to the inputs of $f \diamond g$ consistent with x, such that for some $k' \leq k$:*

- *$(f \diamond g)|_\sigma$ has a parity certificate of size k' at $x|_\sigma$ whose minimum weight is at least K,*
- *$\mathrm{C}\big(f|_{g^n(\sigma)}, g^n(x)|_{g^n(\sigma)}\big) > k'K$. (Note that σ is blockwise and agrees with x, so here we consider the certificate complexity of f at $g^n(x)$ on all the blocks that are not set by σ).*

PROOF. The proof is by induction on k. If $k = 0$, then the premises contradict each other, so we can assume that $k > 0$.

Let C be the parity certificate for $f \diamond g$ at input x. By the assumption, the size of C is at most k. If its minimum weight is at least K, then we are done. Otherwise, without loss of generality, C contains a nontrivial constraint $\ell = \alpha$ whose support is less than K. Choose a partial assignment that agrees with x on the support of this constraint, and complete it to a blockwise partial assignment σ consistent with x (complete only the blocks with at least one variable assigned). Also, let $\tau := g^n(\sigma)$.

By construction, $(f \diamond g)|_\sigma$ has a parity certificate of size at most $k-1$ at $x|_\sigma$, namely $C \setminus \{\ell = \alpha\}$. On the other hand, since we only set the values of at most K blocks, the certificate complexity of f reduces by at most K after being restricted to τ, and hence $\mathrm{C}(f|_\tau, g^n(x)|_\tau) > (k-1)K$. Now we can apply the induction hypothesis for $k-1$. □

The following lemma demonstrates an essential property of the parity certificates with huge minimal weight:

LEMMA 5.4. *Suppose that $C_\oplus$ is a parity certificate with minimum weight at least K and C is a (regular) certificate of size less than K. Then there is a point that satisfies both C and $C_\oplus$.*

In the proof, the parity certificate is processed with a careful Gaussian elimination, and then the variables fixed by the certificate C are free variables of the parity certificate.

PROOF. Let $V + b$ be the affine subspace corresponding to $C_\oplus$, where V is a linear subspace. We pick a basis $\ell_1, \ldots, \ell_k$ for V and indices $i_1, \ldots, i_k \in [n]$ such that:

- z_{i_t} is in the support of ℓ_t,
- z_{i_s} is not in the support of ℓ_t for $s < t$,
- z_{i_t} does not belong to the support of C.

We do it inductively. At step t, we have already picked $\ell_1, \ldots, \ell_{t-1}$. Consider vectors in V whose support does not contain any of the variables $z_{i_1}, \ldots, z_{i_{t-1}}$. Since V has minimum weight K, we can find such a vector ℓ_t whose support contains a variable z_{i_t} with $z_{i_t} \notin C$. By the assumption, this vector does not contain any of the variables $z_{i_1}, \ldots, z_{i_{t-1}}$.

Now we can construct an input α which satisfies both C and $C_\oplus$. First, we set the variables from C such that the resulting input satisfies C. Then we set arbitrarily all the variables not occurring in C or $\{z_{i_1}, \ldots, z_{i_k}\}$. Finally, we set variables $z_{i_k}, \ldots, z_{i_1}$ (in this order) in such a way that the resulting input satisfies $C_\oplus$. □

5.3. OR gadget.

5.3.1. Separation. Chattopadhyay *et al.* (2023) showed that $\mathrm{C}_\oplus(f \diamond g) = \Theta(\mathrm{C}(f))$ whenever g is stifling. This no longer holds when g is binary OR. The following theorem shows that there is at least a quadratic gap for OR gadget in the certificate to parity certificate complexity lifting.

THEOREM 5.5. *Let $n = m^2$, and let $f\colon \{0,1\}^n \to \{0,1\}$ be the function that accepts an $m \times m$ Boolean matrix iff it has no rows of Hamming weight 1. Then:*

(i) $\mathrm{C}(f) = n$,

(ii) $\mathrm{C}_\oplus(f \diamond \mathrm{OR}) \leq 2m$.

PROOF. For the first part, we will show that $\mathrm{C}(f, \vec{0}) = n$. Indeed, $f(\vec{0}) = 1$, but if we flip any bit, we get a 0-input of f.

For the second part, observe first that if $(f \diamond \mathrm{OR})(x) = 0$ then one of the rows has Hamming weight 1, which can be certified using m constraints. We complete the proof by showing that given inputs for one row of the matrix $y, z \in \{0,1\}^m$, we can certify that $y \vee z$ does not have Hamming weight 1 using two constraints.

If $y \vee z$ has at least two 1s, say $(y \vee z)_i = (y \vee z)_j = 1$, then this can be certified using one of y_i, z_i and one of y_j, z_j. If $y = z = \vec{0}$ then we can certify that $y \vee z$ does not have Hamming weight 1 by two constraints $\mathrm{XOR}_n(y) = \mathrm{XOR}_n(z) = 0$. Indeed, if $y \vee z$ had Hamming weight 1, say $y_i = 1$, then $\mathrm{XOR}_n(y) = 1$. □

Note that if we assume that the lifting result of Chattopadhyay *et al.* (2023) holds for the OR gadget as well, then the quadratic separation is tight.

PROPOSITION 5.6. *Suppose that* $\mathrm{DT}_\oplus(f \diamond \mathrm{OR}) = \Theta(\mathrm{DT}(f))$ *for all* f. *Then every function* f *satisfies* $\mathrm{C}(f) = O(\mathrm{C}_\oplus(f \diamond \mathrm{OR})^2)$.

PROOF. The assumption implies that

$$\mathrm{C}(f) \leq \mathrm{DT}(f) = \Theta(\mathrm{DT}_\oplus(f \diamond \mathrm{OR})) = O(\mathrm{C}_\oplus(f \diamond \mathrm{OR})^2),$$

where the last equality is due to Zhang & Shi (2010). □

5.3.2. Lifting. Given Lemma 5.3, we are going to prove the following lifting for the certificate complexity.

THEOREM 5.7. *For every function* $f\colon \{0,1\}^n \to \{0,1\}$, *we have* $\mathrm{C}(f) = O(\mathrm{C}_\oplus(f \diamond \mathrm{OR})^3)$.

Throughout the proof of the theorem, we will think of f as having inputs $x_1, x_2, \ldots, x_n$, and of block composition $f \diamond \mathrm{OR}$ as having inputs $z_{1,1}, z_{1,2}, z_{2,1}, z_{2,2}, \ldots, z_{n,1}, z_{n,2}$, such that x_i corresponds to $z_{i,1} \vee z_{i,2}$. The *twin* of $z_{i,1}$ is $z_{i,2}$, and vice versa. We start the proof with the following lemma, another technical tool in this section. Together with Lemma 5.3, the proof of the lifting theorem is essentially a combination of these two tricks.

LEMMA 5.8. *Suppose that C is a parity certificate for $f \diamond \mathrm{OR}$ of size k whose minimum weight is at least k. Then there are k indices $i_1, \ldots, i_k$ such that for every input x with $x_{i_1} = \cdots = x_{i_k} = 1$, we can find an input z satisfying C such that $\mathrm{OR}^n(z) = x$. In particular, there is an input z satisfying C such that $\mathrm{OR}^n(z) = (1, \ldots, 1)$.*

PROOF. Let $V + b$ be the affine subspace corresponding to C, where V is a linear subspace (the row space of the linear system $Ax = b$ in Definition 5.2). We pick a basis $\ell_1, \ldots, \ell_k$ for V and indices $(i_1, j_1), \ldots, (i_k, j_k) \in [n] \times [2]$ such that:

- z_{i_t, j_t} is in the support of ℓ_t.
- $z_{i_s,1}, z_{i_s,2}$ are not in the support of ℓ_t for $s < t$.

We build the basis by successively adding vectors to it, similar to the Gaussian elimination. At step t, we have already chosen basis vectors $\ell_1, \ldots, \ell_{t-1}$ and the corresponding indices. Consider all the vectors in V whose support does not contain any of the variables $z_{i_1,j_1}, \ldots, z_{i_{t-1},j_{t-1}}$. Since V has minimum weight $k > t - 1$, we can find a vector ℓ_t whose support contains a variable z_{i_t,j_t} with $i_t \notin \{i_1, \ldots, i_{t-1}\}$ and add it to the basis.

Now let x be an input satisfying $x_{i_1} = \cdots = x_{i_k} = 1$. We set the twins of $z_{i_1,j_1}, \ldots, z_{i_t,j_t}$ to 1, and set $z_{i,1}, z_{i,2}$ for $i \neq i_1, \ldots, i_t$ in an arbitrary way subject to $z_{i,1} \vee z_{i,2} = x_i$. We then set $z_{i_t,j_t}, \ldots, z_{i_1,j_1}$ (in this order) in such a way that the resulting input satisfies C. □

The next lemma that we need for the proof of the lifting result gives an upper bound on the minimum weight of a parity certificate in terms of the sizes of certificates for 0 and 1.

LEMMA 5.9. *Suppose that $f' \diamond \mathrm{OR}$ has a parity certificate C' of size k' which certifies b, and a parity certificate C'' of size k'' which certifies $\bar{b}$. Then the minimum weight of C' is at most $k' + 2(k'')^2$.*

PROOF. First, observe that both $k' > 0$ and $k'' > 0$ since otherwise $f' \diamond \mathrm{OR}$ is constant and cannot have certificates for both 0 and 1.

Consider the case $k'' = 1$. The certificate C'' consists of one constraint $\ell = \alpha$. By Lemma 5.8, since C'' has size 1, some input z satisfying C'' satisfies $\mathrm{OR}^n(z) = (1, \ldots, 1)$, and so $f'(1, \ldots, 1) = \bar{b}$. Suppose for the sake of contradiction that the minimum weight of C' is at least $k' + 2(k'')^2 + 1 = k' + 3 > k'$. According to Lemma 5.8, some input z satisfying C' satisfies also $\mathrm{OR}^n(z) = (1, \ldots, 1)$, and so $f'(1, \ldots, 1) = b$, which is impossible. So the minimum weight of C' is at most $k' + 2$.

We proceed by induction on k''. Suppose for the sake of contradiction that the minimum weight of C' is at least $k' + 2(k'')^2 + 1 > k'$. According to Lemma 5.8, some input z satisfying C' satisfies $\mathrm{OR}^n(z) = (1, \ldots, 1)$, and so $f'(1, \ldots, 1) = b$. If the minimum weight of C'' was also at least k'', then the same argument would imply that $f'(1, \ldots, 1) = \bar{b}$. Therefore, the minimum weight of C' is less than k''. Consider some constraint $\ell = \alpha$ of C'' of weight at most k''. Lets choose some partial assignment satisfying $\ell = \alpha$, and complete it to a blockwise partial assignment τ in an arbitrary way. Thus τ assigns at most $2k''$ variables. By Lemma 5.4, the assignment τ cannot contradict C' since the minimum weight of C' is greater than $k' + 2(k'')^2 > 2k''$. Therefore $(f' \diamond \mathrm{OR})|_\tau$ still has a b-certificate $C'|_\tau$ of size k' and a $\bar{b}$-certificate $C''|_\tau$ of size $k'' - 1$. By induction, we know that the minimum weight of $C'|_\tau$ is at most $k' + 2(k'' - 1)^2$, and so the minimum weight of C' is at most $k' + 2(k'' - 1)^2 + 2k'' < k' + 2(k'')^2$. □

The following lemma applies the 'minimal weight lemma' to the current setting.

Lemma 5.10. *Suppose that $f \diamond \mathrm{OR}$ has a parity certificate at some input z of size at most k. Let K be a parameter. Suppose that the certificate complexity of f at $\mathrm{OR}^n(z)$ is greater than $k \cdot K$. Then we can find a blockwise partial assignment σ to the inputs of $f \diamond \mathrm{OR}$, consistent with z, such that for some $k' \leq k$:*

- *$(f \diamond \mathrm{OR})|_\sigma$ has a parity certificate of size k' at $z|_\sigma$ whose minimum weight is at least K.*
- *$\mathrm{C}(f|_{\mathrm{OR}^n(\sigma)}, \mathrm{OR}^n(z)|_{\mathrm{OR}^n(\sigma)}) > k'K$.*

Proof. We apply Lemma 5.3 for a function f and a gadget $g = \mathrm{OR}$. □

Now we have all the ingredients for proving Theorem 5.7.

Proof (Proof of Theorem 5.7). Let $\mathrm{C}_{\oplus}(f \diamond \mathrm{OR}) = k$, and let K be a parameter to be determined later. Suppose that $\mathrm{C}(f) > kK$. Apply Lemma 5.10 to get a non-constant function f', obtained by restricting f, such that $f' \diamond \mathrm{OR}$ has a parity certificate of size k' whose minimum weight is at least K (the function f' is non-constant since its certificate complexity is positive). Since $\mathrm{C}_{\oplus}(f' \diamond \mathrm{OR}) \leq \mathrm{C}_{\oplus}(f \diamond \mathrm{OR})$ and f' is non-constant, Lemma 5.9 (with $k'' \leq k$) shows that $K \leq k' + 2(k'')^2 \leq k + 2k^2$. By choosing $K = k + 2k^2 + 1$ we reach a contradiction. Therefore $\mathrm{C}(f) \leq kK = O(k^3)$. □

Question 5.11. *Can we improve this bound from cubic to quadratic so that this bound matches the separation provided by Proposition 5.6?*

5.4. AND/OR **gadgets.** A function $g\colon \{0,1\}^m \to \{0,1\}$ *affine projects* to a function $h\colon \{0,1\}^p \to \{0,1\}$ if there are affine functions $\ell_1, \ldots, \ell_m\colon \mathbb{Z}_2^p \to \mathbb{Z}_2$ such that

$$g(\ell_1(z), \ldots, \ell_m(z)) = h(z).$$

A gadget $g\colon \{0,1\}^m \to \{0,1\}$ is AND/OR if it affine projects to both binary AND and binary OR. Here are some examples of AND/OR gadgets:

- $g(x,y,z) := x \vee (y \wedge z)$. The projections: $g(0,a,b) = a \wedge b$ and $g(a,b,1) = a \vee b$.

- $g(x,y,z) := [x+y+z = 1]$. The projections: $g(1,\bar{a},\bar{b}) = a \wedge b$ and $g(\bar{a},\bar{b},\overline{a \oplus b}) = a \vee b$.

THEOREM 5.12. *If g is an* AND/OR *gadget then for all functions $f\colon \{0,1\}^n \to \{0,1\}$*

$$\log \mathrm{DTSize}_{\oplus}(f \diamond g) \geq \mathrm{C}(f).$$

PROOF. Let T_0 be a parity decision tree for $f \diamond g$. Let z be a point such that $\mathrm{C}(f,z) = \mathrm{C}(f)$. Since g is AND/OR, we can apply affine substitutions to T_0 and get a parity decision tree T computing function $f \diamond (h_{z_1}, \ldots, h_{z_n})$, where $h_0(a,b) := a \wedge b$ and $h_1(a,b) := a \vee b$.

Now we follow the simulation proof of Chattopadhyay *et al.* (2023). We start at the root. When at a node v, if the query is constant then we follow the corresponding edge, otherwise we pick some variable, without loss of generality a_i, which appears in the query. So the query is of the form $a_i \oplus \ell$. We set $b_i := 0$ if $z_i = 0$, and we set $b_i := 1$ if $z_i = 1$. Then we choose c, the answer to the query, such that it leads to the smaller subtree, and set $a_i := \ell \oplus c$. From now on, we see neither a_i nor b_i.

Eventually, we reach a leaf. Suppose that this happens after eliminating coordinates $i_1, \ldots, i_K$, which means that T contains at least 2^K leaves. For every input z' to f which agrees with z on $i_1, \ldots, i_K$ we can find an input w' to T which satisfies $g(w') = z'$ and leads to the same leaf, showing that $z'_{i_1} = z_{i_1}, \ldots, z'_{i_K} = z_{i_K}$ is a certificate for f at z. We conclude that $K \geq \mathrm{C}(f,z) = \mathrm{C}(f)$, and so $|T_0| \geq |T| \geq 2^K$. □

5.5. Gadget classification. For the proof, we will need the following classical fact.

PROPOSITION 5.13 (Chen 2009, Theorem 4.3). *Suppose that $f\colon \{0,1\}^n \to \{0,1\}$ satisfies the following condition: whenever $f(x) = f(y) = f(z) = 1$ then $f(x \oplus y \oplus z) = 1$. Then either $f \equiv 0$, or f is an indicator of an affine subspace.*

We need this lemma to classify gadgets that are not AND/OR.

LEMMA 5.14. *If g is not an* AND/OR *then g is a disjunction or a conjunction of affine forms in the inputs.*

Proof. Suppose that g does not affine project to OR. We claim that for any inputs x, y, z, if $g(x) = g(y) = g(z) = 1$ then $g(x \oplus y \oplus z) = 1$. Indeed, suppose that $g(x) = g(y) = g(z) = 1$ but $g(x \oplus y \oplus z) = 0$, and consider the projection

$$h(a, b) := g(x \oplus y \oplus z \oplus a(x \oplus y) \oplus b(x \oplus z)).$$

Then $h(0, 0) = g(x \oplus y \oplus z) = 0$, $h(1, 0) = g(z) = 1$, $h(0, 1) = g(y) = 1$, and $h(1, 1) = g(x) = 1$. Thus, we can conclude that g affine projects to OR.

Applying Proposition 5.13, we see that if g does not affine project to OR then it is either the empty disjunction or a conjunction of affine forms. Similarly, if g does not affine project to AND then $\bar{g}$ does not affine project to OR, and so g is either the empty conjunction or a disjunction of affine forms. □

Now we can prove the classification of gadgets.

Theorem 5.15. *For every gadget g, one of the following cases holds:*

(i) There is an infinite family of functions f_n with $\mathrm{DT}(f_n) \to \infty$ and $\mathrm{DTSize}_{\oplus}(f_n \diamond g) = O(1)$.

(ii) For every function f, $\mathrm{DT}_{\oplus}(f \diamond g) = \Omega\left(\mathrm{DT}(f)^{\Omega(1)}\right)$. There is an infinite family of functions f_n with $\mathrm{DT}(f_n) \to \infty$ and $\mathrm{DTSize}(f_n \diamond g) = O(\mathrm{DT}(f))$.

(iii) For every function f, $\log \mathrm{DTSize}_{\oplus}(f \diamond g) = \Omega\left(\mathrm{DT}(f)^{\Omega(1)}\right)$ and also $\mathrm{C}_{\oplus}(f \diamond g) = \Omega\left(\mathrm{C}(f)^{\Omega(1)}\right)$.

Proof. If gadget g is AND/OR then by Theorem 5.12 we have

$$\log \mathrm{DTSize}_{\oplus}(f \diamond g) \geq \mathrm{C}(f) = \Omega\left(\sqrt{\mathrm{DT}(f)}\right).$$

Since $\mathrm{C}_{\oplus}(f \diamond g) = \Omega\left(\sqrt{\mathrm{DT}_{\oplus}(f \diamond g)}\right)$ (Zhang & Shi 2010) and also $\mathrm{DT}_{\oplus}(f \diamond g) \geq \log \mathrm{DTSize}_{\oplus}(f \diamond g)$, we get the third case of the theorem.

Otherwise, by Lemma 5.14, gadget g is either a conjunction of affine forms or a disjunction of affine forms. Suppose, without loss

of generality, that g is a disjunction of affine forms. If $g \equiv 0$ or g is an affine form, then we can take $f_n = \oplus_n$ to get the first case of the theorem. If g is not affine then it affine projects to OR, and thus we get the second case: we need to apply Theorem 5.7 together with the polynomial relations between (parity) certificate complexity and (parity) decision tree complexity; the infinite family of functions is $f_n = \mathrm{OR}_n$. □

5.6. Lifting for OR gadget and the log-rank conjecture. In this section, we provide a proof of the following inequality:

THEOREM 5.16. *For any function* $f\colon \{0,1\}^n \to \{0,1\}$,

$$\mathrm{DT}(f) \leq O\left(\mathrm{DT}_{\oplus}(f \diamond \mathrm{OR})^{O(1)}\right).$$

This inequality is a corollary of Theorem 5.7 using the fact that DT is polynomially related to C, and $\mathrm{DT}_{\oplus}$ is polynomially related to $\mathrm{C}_{\oplus}$. We present an alternative proof of this statement that gives as a byproduct a proof of the log-rank conjecture for a subclass of Boolean functions that can be represented as $f \diamond \mathrm{OR} \diamond \mathrm{XOR}$ for an arbitrary function f.

In this section, we will need a composition of a function and a gadget (XOR in this case) in the communication complexity context. For a function $f\colon \{0,1\}^n \to \{0,1\}$ we define a function $f_{\oplus}\colon \{0,1\}^n \times \{0,1\}^n \to \{0,1\}$, such that $f_{\oplus}(x,y) := f(x_1 \oplus y_1, \ldots, x_n \oplus y_n)$, and associate it with the following communication problem: Alice and Bob are given x and y, respectively, and their goal is to compute $f(x,y)$. For a subclass of such XOR functions, we will prove the following version of log-rank conjecture:

THEOREM 5.17. *For any function* $f\colon \{0,1\}^n \to \{0,1\}$,

$$\mathrm{D}((f \diamond \mathrm{OR})_{\oplus}) \leq \mathrm{poly}(\log \mathrm{rank}((f \diamond \mathrm{OR})_{\oplus}).$$

The proof depends on the following lemma.

LEMMA 5.18. *For any function* $f\colon \{0,1\}^n \to \{0,1\}$,

$$\mathrm{DT}(f) \overset{(1)}{\leq} 2\deg(f)^4 \overset{(2)}{\leq} O((\log \mathrm{spar}(f \diamond \mathrm{OR}))^4)$$
$$\overset{(3)}{=} O((\log \mathrm{rank}((f \diamond \mathrm{OR})_{\oplus}))^4) \overset{(4)}{\leq} O(\mathrm{D}((f \diamond \mathrm{OR})_{\oplus})^4)$$
$$\overset{(5)}{\leq} O(\mathrm{DT}_{\oplus}(f \diamond \mathrm{OR})^4).$$

PROOF.

(1) See Buhrman & de Wolf (2002).

(2) We need to prove that

$$\deg(f) \leq \log \mathrm{spar}(f \diamond \mathrm{OR}).$$

We can assume that $f\colon \{-1,1\}^n \to \{-1,1\}$ has degree n. If not, we can fix a monomial of the maximum degree and substitute 1 into all other variables. This substitution does not change the degree, while the sparsity can only decrease. So, the function f has the following multilinear representation:

$$f(x) = \alpha_{[n]} \prod_{i=1}^{n} x_i + \sum_{S \subsetneq [n]} \alpha_S x_S, \qquad \text{where } \alpha_{[n]} \neq 0.$$

In order to get the multilinear representation of the function $f \diamond \mathrm{OR}$ one can substitute each x_i with $\frac{y_{i1}y_{i2} - y_{i1} - y_{i2} - 1}{2}$. After the substitution, for any combination of indices $j_1, j_2, \ldots, j_n \in \{1,2\}$, the coefficient of the monomial $\prod_{k=1}^{n} y_{kj_k}$ is equal to $\alpha_{[n]} \cdot 2^{-n}$. Indeed, such monomials can only be produced by the substitution into the maximum degree monomial, and such substitution results in the coefficient $\alpha_{[n]} \cdot 2^{-n}$.

Note that $\alpha_{[n]} \neq 0$ and hence there is a monomial for every 2^n possible combinations of indexes $j_1, j_2, \ldots, j_n$. That gives us the desired bound:

$$\log \mathrm{spar}(f \diamond \mathrm{OR}) \geq n = \deg(f).$$

(3) See Bernasconi & Codenotti (1999).

(4) See Lovsz & Saks (1993).

(5) Each time we query a linear function $\ell(x \oplus y) = \ell(x) \oplus \ell(y)$, Alice and Bob can send to each other $\ell(x)$ and $\ell(y)$ to compute $\ell(x \oplus y)$. □

Theorem 5.16 follows immediately. For Theorem 5.17, we need one more inequality in the chain.

Proof (Theorem 5.17). We need to show that

$$\mathrm{DT}_{\oplus}(f \diamond \mathrm{OR}) \leq 2\,\mathrm{DT}(f).$$

Indeed, the decision tree for f can be transformed into a decision tree for $f \diamond \mathrm{OR}$ by replacing every query with two queries of corresponding variables. Together with Lemma 5.18 that finishes the proof. □

6. Block sensitivity to communication complexity

In this section, we provide a complete classification of gadgets for lifting from block sensitivity to deterministic and randomized communication complexities. Note that this classification will also give us a classification of gadgets for query-to-communication polynomial lifting since block sensitivity and query complexity are polynomially related to each other in the case of total functions. However, since the proof goes through block sensitivity, we will classify block sensitivity to communication complexity lifting.

6.1. Reductions in communication complexity. Consider a pair of two-party functions $f_i \colon \mathcal{X}_i \times \mathcal{Y}_i \to \{0,1\}$ for $i = 1, 2$. We say that f_1 *reduces to* f_2, denoted $f_1 \leq f_2$, if the communication matrix of f_1 is a submatrix of the communication matrix of f_2. Equivalently, $f_1 \leq f_2$ iff there exist one-to-one mappings π_A and π_B such that

$$f_1(x, y) = f_2(\pi_A(x), \pi_B(y)), \quad \forall (x, y) \in \mathcal{X}_1 \times \mathcal{Y}_1.$$

Zhang (2009) proved the following theorem:

THEOREM 6.1 (Zhang). *If a two-party gadget $g\colon \mathcal{X} \times \mathcal{Y} \to \{0,1\}$ satisfies* $\mathrm{AND}, \mathrm{OR} \leq g$, *then for every function $f\colon \{0,1\}^n \to Q$, the function $f \diamond g$ has (constant error) randomized communication complexity $\Omega(\mathrm{bs}(f))$.*

For completeness, we present the proof that is a reduction from unique set disjointness.

DEFINITION 6.2. *The* disjointness *function* $\mathsf{Disj}_n\colon \{0,1\}^n \times \{0,1\}^n$ *is defined as follows:*

$$\mathsf{Disj}_n(x,y) := \bigvee_{i=1}^{n} x_i \wedge y_i.$$

Unique set disjointness, $\mathsf{PromiseDisj}_n$, *is a promise version of disjointness with the promise that there is always at most one i such that $x_i \wedge y_i = 1$.*

PROOF (Proof of Theorem 6.1). Razborov (1992) showed that $\mathrm{R}(\mathsf{PromiseDisj}_m) = \Omega(m)$. We will show a reduction from this problem to computing $f \diamond g$. To illustrate the proof, let us start with the case in which $f(\vec{0}) = 0$ and $f(e_i) = 1$ for all $i \in [n]$, where e_i is the i'th unit vector. Then

$$\mathsf{PromiseDisj}_n(x,y) = f(x_1 \wedge y_1, \ldots, x_n \wedge y_n).$$

Since $\mathrm{AND} \leq g$, we can solve unique set disjointness using a protocol for $f \diamond g$. Therefore, $f \diamond g$ has communication complexity $\Omega(n)$. Note that this case only required $\mathrm{AND} \leq g$; we need $\mathrm{OR} \leq g$ in order to handle inputs changing from 1 to 0.

To handle the general case, let inputs $X^\wedge(0), X^\wedge(1) \in \mathcal{X}$ and $Y^\wedge(0), Y^\wedge(1) \in \mathcal{Y}$ witness $\mathrm{AND} \leq g$, and let $X^\vee(0), X^\vee(1) \in \mathcal{X}$ and $Y^\vee(0), Y^\vee(1) \in \mathcal{Y}$ witness $\mathrm{OR} \leq g$.

Let z be a point witnessing $\mathrm{bs}(f)$, and let $B_1, \ldots, B_m$ be the corresponding blocks, where $m = \mathrm{bs}(f)$. Given inputs x, y to unique set disjointness, we construct the following inputs X, Y to $f \diamond g$:

- If i does not belong to any of the blocks, then we fix X_i, Y_i so that $g(X_i, Y_i) = z_i$.

- If $i \in B_j$ and $z_i = 0$, put $X_i = X^{\wedge}(x_j)$, $Y_i = Y^{\wedge}(y_j)$.
- If $i \in B_j$ and $z_i = 1$, put $X_i = X^{\vee}(\bar{x}_j)$, $Y_i = Y^{\vee}(\bar{y}_j)$.

The reader can check that if x, y are disjoint, then applying g to X, Y yields z, and if they intersect only at j, then it yields $z \oplus B_j$. Hence, $f \diamond g$ can be used to compute unique set disjointness of length m, and consequently, we obtain a lower bound of $\Omega(m)$. □

For the classification of gadgets, we will need the following lemma that shows that if OR $\not\leq g$ then the communication matrix of g is block diagonal.

LEMMA 6.3. *If $g\colon \mathcal{X} \times \mathcal{Y} \to \{0,1\}$ satisfies that* OR $\not\leq g$ *then there are partitions of $\mathcal{X}$ and $\mathcal{Y}$ into* disjoint *sets*

$$\mathcal{X} = \mathcal{X}_0 \sqcup \mathcal{X}_1 \sqcup \cdots \sqcup \mathcal{X}_k \quad \text{and} \quad \mathcal{Y} = \mathcal{Y}_0 \sqcup \mathcal{Y}_1 \sqcup \cdots \sqcup \mathcal{Y}_k,$$

such that

- *If $x \in \mathcal{X}_i, y \in \mathcal{Y}_j$, where $i \neq j$, or $i = 0$, or $j = 0$, then $g(x,y) = 0$.*
- *If $x \in \mathcal{X}_i, y \in \mathcal{Y}_i$, where $i \neq 0$, then $g(x,y) = 1$.*

PROOF. We can construct such a partition step by step. Consider any point (x_0, y_0) such that $g(x_0, y_0) = 1$. If we don't have such a point, then we can take $\mathcal{X}_0 = \mathcal{X}$ and $\mathcal{Y}_0 = \mathcal{Y}$. Otherwise, we consider all points $y \in \mathcal{Y}$ such that $g(x_0, y) = 1$. Denote the set of all such points as $\mathcal{Y}'$. Now, we can consider all $x \in \mathcal{X}$ such that $g(x, y_0) = 1$. Denote the set of all such points as $\mathcal{X}'$. Observe that every point $(x, y) \in \mathcal{X}' \times \mathcal{Y}'$ satisfies $g(x, y) = 1$, otherwise we can project our gadget to points $(x_0, y_0), (x_0, y), (x, y_0), (x, y)$ and get OR.

Now, observe that for any $(x'', y') \in (\mathcal{X} \setminus \mathcal{X}') \times \mathcal{Y}'$ we have $g(x'', y_0) = 0$ by construction, and thus $g(x'', y') = 0$, since otherwise we can project the gadget to points (x_0, y'), (x_0, y_0), (x'', y'), (x'', y_0) and get OR. By analogy, for any $(x', y'') \in \mathcal{X}' \times (\mathcal{Y} \setminus \mathcal{Y}')$ we have $g(x', y'') = 0$.

This means that $\mathcal{X}' \times \mathcal{Y}'$ forms a valid block of our partition, so we can use the same procedure to construct a block of $(\mathcal{X} \setminus$

$\mathcal{X}') \times (\mathcal{Y} \setminus \mathcal{Y}')$. In the end, if we get sets $\mathcal{X}''$ and $\mathcal{Y}''$ such that any $(x, y) \in \mathcal{X}'' \times \mathcal{Y}''$ satisfies $g(x, y) = 0$, we take $\mathcal{X}_0 = \mathcal{X}''$ and $\mathcal{Y}_0 = \mathcal{Y}''$. □

Definition 6.4. *We say that a gadget* $g\colon \mathcal{X} \times \mathcal{Y} \to \{0,1\}$ *is a* blow-up *of a gadget* $h\colon \mathcal{Z} \times \mathcal{W} \to \{0,1\}$ *if there are decompositions* $\mathcal{X} = \bigsqcup_{z \in \mathcal{Z}} \mathcal{X}_z$ *and* $\mathcal{Y} = \bigsqcup_{w \in \mathcal{W}} \mathcal{Y}_w$*, with* $\mathcal{X}_z, \mathcal{Y}_w \neq \emptyset$*, such that all* $(x_z, y_w) \in \mathcal{X}_z \times \mathcal{Y}_w$ *satisfy* $g(x_z, y_w) = h(z, w)$.

For example, g is a blow-up of XOR if (up to rearrangement) it has a communication matrix of the following form

$$\begin{bmatrix} 1 & \cdots & 1 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 1 & \cdots & 1 & 0 & \cdots & 0 \\ 0 & \cdots & 0 & 1 & \cdots & 1 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & 1 & \cdots & 1 \end{bmatrix}$$

If g is a blow-up of h, then $f \diamond g$ and $f \diamond h$ have the same communication complexity in all models. Indeed, on the one hand, h is a restriction of g, and so a protocol for $f \diamond g$ can be used to solve $f \diamond h$; and on the other hand, by replacing $x \in \mathcal{X}_z$ by z and $y \in \mathcal{Y}_w$ by w, we can use a protocol for $f \diamond h$ to solve $f \diamond g$.

Corollary 6.5. *If* $g\colon \mathcal{X} \times \mathcal{Y} \to \{0,1\}$ *satisfies both* OR $\not\leq g$ *and* AND $\not\leq g$ *then either* g *is constant, or it is a blow-up of* XOR.

Proof. Let $\mathcal{X} = \mathcal{X}_0 \sqcup \mathcal{X}_1 \sqcup \cdots \sqcup \mathcal{X}_k$ and $\mathcal{Y} = \mathcal{Y}_0 \sqcup \mathcal{Y}_1 \sqcup \cdots \sqcup \mathcal{Y}_k$ be the decomposition provided by Lemma 6.3. We can assume that the $\mathcal{X}_i, \mathcal{Y}_i \neq \emptyset$, except that possibly $\mathcal{X}_0 = \mathcal{Y}_0 = \emptyset$.

If $\mathcal{X}_0 \neq \emptyset$ and $k > 0$ then AND $\leq g$. Indeed, if we restrict the matrix of g so that it contains one row each from $\mathcal{X}_0, \mathcal{X}_1$ and one column each from $\mathcal{Y}_0, \mathcal{Y}_1$ then we get

$$\begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$$

which is AND. Consequently, either $k = 0$ or $\mathcal{X}_0 = \mathcal{Y}_0 = \emptyset$.

If $k = 0$ then $g \equiv 0$. Now suppose that $\mathcal{X}_0 = \mathcal{Y}_0 = \emptyset$. If $k \geq 3$ and we restrict the communication matrix of g so that it contains one row from each $\mathcal{X}_1, \mathcal{X}_2, \mathcal{X}_3$ and one column from each $\mathcal{Y}_1, \mathcal{Y}_2, \mathcal{Y}_3$ then we get

$$\begin{bmatrix} 1 & \mathbf{0} & \mathbf{0} \\ 0 & 1 & 0 \\ 0 & \mathbf{0} & \mathbf{1} \end{bmatrix}$$

Again, we get a restriction to AND (the entries in bold text correspond to the AND matrix). Hence, $k \leq 2$. If $k = 1$ then $g \equiv 1$, and otherwise g is a blow-up of XOR. □

COROLLARY 6.6. *If $g\colon \mathcal{X} \times \mathcal{Y} \to \{0,1\}$ satisfies both* OR $\not\leq g$ *and* XOR $\not\leq g$ *then either g is constant, or it is a blow-up of* AND. *Similarly, if* AND $\not\leq g$ *and* XOR $\not\leq g$ *then either g is constant, or it is a blow-up of* OR.

PROOF. We can assume that the $\mathcal{X}_i, \mathcal{Y}_i \neq \emptyset$, except that possibly $\mathcal{X}_0 = \mathcal{Y}_0 = \emptyset$. We start by proving the first claim, and so we assume that $\mathrm{OR}, \mathrm{XOR} \not\leq g$.

If $k \geq 2$ then XOR $\leq g$. Indeed, if we restrict the matrix of g so that it contains one row from each $\mathcal{X}_1, \mathcal{X}_2$ and one column from each $\mathcal{Y}_1, \mathcal{Y}_2$ then we obtain

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

which is XOR. If $k = 0$ then $g \equiv 0$. If $k = 1$ and $\mathcal{X}_0 = \mathcal{Y}_0 = \emptyset$ then $g \equiv 1$. Finally, if $k = 1$ and $\mathcal{X}_0, \mathcal{Y}_0 \neq \emptyset$ then g is a blow-up of AND.

Suppose that $\mathrm{AND}, \mathrm{XOR} \not\leq g$. Then $\mathrm{OR}, \mathrm{XOR} \not\leq \bar{g}$, where $\bar{g}$ is the negation of g. Hence, either $\bar{g}$ is constant or it is a blow-up of AND. Thus, either g is constant or it is a blow-up of OR. □

6.2. Gadget classification for randomized communication complexity. In this section, we prove the following dichotomy result for the randomized case.

THEOREM 6.7. *For every gadget g, one of the following cases holds:*

(i) There is an infinite family of functions f_n with $\mathrm{bs}(f_n) = \Omega(n)$ and $\mathrm{R}(f_n \diamond g) = O(\log n)$.

(ii) For every function f, we have $\mathrm{R}(f \diamond g) = \Omega\left(\mathrm{bs}(f)^{\Omega(1)}\right)$.

First, we need to prove the following lemma.

LEMMA 6.8. *Suppose that gadgets $g, h\colon \mathcal{X} \times \mathcal{Y} \to \{0,1\}$ are such that $\mathrm{OR} \not\leq g$ and $\mathrm{AND} \not\leq h$. Then*

$$\mathrm{R}(\mathrm{AND}_n \diamond g) = O(\log n), \quad \mathrm{R}(\mathrm{OR}_n \diamond h) = O(\log n).$$

PROOF. The bounds for g and h are symmetrical, so we prove only the first one. Consider the decomposition $\mathcal{X} = \mathcal{X}_0 \sqcup \mathcal{X}_1 \sqcup \cdots \sqcup \mathcal{X}_k$ and $\mathcal{Y} = \mathcal{Y}_0 \sqcup \mathcal{Y}_1 \sqcup \cdots \sqcup \mathcal{Y}_k$ provided by Lemma 6.3. If one of the blocks g_i was given an input from $\mathcal{X}_0$ or $\mathcal{Y}_0$, then one of the players can share this information with the other player and finish the communication. Otherwise, they indicate that they do not have such blocks. After this step, they are restricted to inputs from $\mathcal{X} \setminus \mathcal{X}_0$ and $\mathcal{Y} \setminus \mathcal{Y}_0$. Since any two inputs from the same block $\mathcal{X}_i$ or $\mathcal{Y}_i$ are indistinguishable for our gadget, they need to solve $\mathrm{AND}_n \diamond \mathrm{EQ}_m$, where EQ_m is the *equality* function for m-bit inputs (i.e., $\mathrm{EQ}_m(x,y) = 1 \iff x = y$) for $m = \lceil \log k \rceil$. This is exactly an instance of EQ_{nm}, which can be solved with $O(\log n)$ bits of communication (note that m is a constant) by a randomized communication protocol (see Kushilevitz & Nisan (1996)). □

Now we are ready to prove the dichotomy result.

PROOF (Theorem 6.7). If $\mathrm{AND}, \mathrm{OR} \leq g$ then Theorem 6.1 shows that $\mathrm{R}(f \diamond g) = \Omega\left(\mathrm{bs}(f)^{\Omega(1)}\right)$, so we get the second case.

Otherwise, $\mathrm{OR} \not\leq g$ or $\mathrm{AND} \not\leq g$. We apply Lemma 6.8: if $\mathrm{OR} \not\leq g$ we take $f_n = \mathrm{AND}_n$, otherwise we take $f_n = \mathrm{OR}_n$. This gives us the first case of the theorem. □

6.3. Gadget classification for deterministic communication complexity. In this section, we will prove the following lifting theorem that implies the classification theorem.

THEOREM 6.9. *If a two-party gadget* $g\colon \mathcal{X} \times \mathcal{Y} \to \{0,1\}$ *satisfies* $\mathrm{AND}, \mathrm{XOR} \leq g$*, then for every function* $f\colon \{0,1\}^n \to \{0,1\}$*, the function* $f \diamond g$ *has communication complexity* $\Omega(\mathrm{bs}(f)^k)$ *for some fixed constant* $k > 0$*. The same is true if* g *satisfies* $\mathrm{OR}, \mathrm{XOR} \leq g$*.*

Together with Theorem 6.1, this will give us a complete classification of gadgets.

THEOREM 6.10. *For every gadget* g*, one of the following cases holds:*

(i) There is an infinite family of functions f_n *with* $\mathrm{bs}(f_n) = \Omega(n)$ *and* $\mathrm{D}(f_n \diamond g) = O(1)$*.*

(ii) For every function f*, we have* $\mathrm{D}(f \diamond g) = \Omega\left(\mathrm{bs}(f)^{\Omega(1)}\right)$*.*

PROOF. If $\mathrm{AND}, \mathrm{OR} \leq g$, or $\mathrm{AND}, \mathrm{XOR} \leq g$, or $\mathrm{OR}, \mathrm{XOR} \leq g$, then by Theorem 6.1 and Theorem 6.9, $\mathrm{D}(f \diamond g) = \Omega\left(\mathrm{bs}(f)^{\Omega(1)}\right)$, so the second case holds.

If none of these cases hold, then at least two of the functions $\mathrm{AND}, \mathrm{OR}, \mathrm{XOR}$ do not reduce to g. Corollary 6.5 and Corollary 6.6 show that either g is constant (and so the first case trivially holds) or it is a blow-up of one of the functions $\mathrm{XOR}, \mathrm{OR}, \mathrm{AND}$. By taking $f_n = \mathrm{XOR}_n, \mathrm{OR}_n, \mathrm{AND}_n$ (respectively), we get the first case of the theorem. □

In the rest of this section, we will cover the proof of Theorem 6.9. We start with the following observation: according to Hatami *et al.* (2018), for any $f\colon \{0,1\}^n \to \{0,1\}$ we have $\mathrm{DT}_\oplus(f) \leq O(\mathrm{D}(f \diamond \mathrm{XOR})^6)$. Zhang & Shi (2010) showed that $\mathrm{DT}_\oplus(f) = O(\mathrm{C}_\oplus(f)^2)$. Combining these results, we can derive the following corollary:

COROLLARY 6.11. *For any* $f\colon \{0,1\}^n \to \{0,1\}$*, we have*

$$\mathrm{C}_\oplus(f) \leq O(\mathrm{D}(f \diamond \mathrm{XOR})^{12}).$$

Using this corollary, our main goal will be to find a substitution τ and two points x and y such that $f(x) \neq f(y)$ and $x|_\tau$ and $y|_\tau$ have parity certificates of large minimal weight. Then, using Lemma 6.12 and Lemma 6.13, we will get the required lifting.

In the following lemma, a *minimal sensitive block* of a function $f\colon \{0,1\}^n \to \{0,1\}$ at point x is a subset $I \subseteq [n]$ such that $f(x^I) \neq f(x)$ but $f(x^J) = f(x)$ for all $J \subsetneq I$. This lemma shows that the deterministic communication complexity of $f \diamond \mathrm{XOR}$ is lower bounded by the size of any minimal sensitive block of f.

LEMMA 6.12. *For any $f\colon \{0,1\}^n \to \{0,1\}$ and $\alpha \in \{0,1\}^n$ such that there is a minimal sensitive block of f at α of size S, the following holds:*

$$\mathrm{D}(f \diamond \mathrm{XOR}) \geq S.$$

PROOF. Consider some minimal sensitive block of f at α of size S. Let $i_1, i_2, \ldots, i_S$ be the indices of the variables in this block of α. Let us restrict the function f by substituting values $x_j = \alpha_j$ for all $j \notin \{i_1, i_2, \ldots, i_S\}$. This gives us a function $f'(y_1, y_2, \ldots, y_S)$ such that

- $f'(y_1, \ldots, y_S) = f(\alpha)$, if $y_j = \alpha_{i_j}$ for at least one j.
- $f'(y_1, \ldots, y_S) \neq f(\alpha)$, if $y_j \neq \alpha_{i_j}$ for all j.

Thus, if we substitute XOR inside each y_j this will give us an instance of the function

$$(z_1 \oplus z'_1 \oplus \alpha_{i_1}) \vee (z_2 \oplus z'_2 \oplus \alpha_{i_2}) \vee \cdots \vee (z_S \oplus z'_S \oplus \alpha_{i_S}),$$

where each z_i and z'_i are distinct variables. This function is essentially a negation of the equality function with a shift. So, its deterministic complexity equals S (see Kushilevitz & Nisan (1996) for the lower bound for EQ_n). Thus,

$$\mathrm{D}(f \diamond \mathrm{XOR}) \geq \mathrm{D}(f' \diamond \mathrm{XOR}) \geq S. \qquad \square$$

The following lemma shows that the deterministic communication complexity of $f \diamond \mathrm{AND}$ is lower bounded by the block sensitivity of f at the all-0 input.

LEMMA 6.13. *For any* $f\colon \{0,1\}^n \to \{0,1\}$ *such that* $\mathrm{bs}(f,\vec{0}) = S$, *the following holds:*

$$\mathrm{D}(f \diamond \mathrm{AND}) \geq \Omega(S).$$

PROOF. The proof of the lemma essentially follows the proof from Zhang (2009). Let $I_1, \ldots, I_S$ be the minimal sensitive blocks of f at $\vec{0}$. We describe a reduction from $\mathsf{PromiseDisj}_S$ to $f \diamond \mathrm{AND}$, so any protocol for $f \diamond \mathrm{AND}$ can be used to solve $\mathsf{PromiseDisj}_S$, and thus $\mathrm{D}(f \diamond \mathrm{AND}) \geq \Omega(S)$. For any input $(a,b) \in \{0,1\}^S$ to $\mathsf{PromiseDisj}_S$ we define the corresponding input $(x,y) \in \{0,1\}^n$ to $f \diamond \mathrm{AND}$.

1. Let $x_i = y_i = 0$ for all $i \notin \bigcup_{j=1}^{S} I_j$.
2. Let $x_i = a_j$ and $y_i = b_j$ for all $i \in I_j$.

We claim that this is a correct reduction. Indeed, if there is a unique j such that $a_j \wedge b_j = 1$, then

$$f(x_1 \wedge y_1, x_2 \wedge y_2, \ldots, x_n \wedge y_n) = f\left(\mathbf{0}^{I_j}\right) \neq f(0,\ldots,0).$$

On the other hand, if $a_j \wedge b_j = 0$ for all $j \in [S]$, then

$$f(x_1 \wedge y_1, x_2 \wedge y_2, \ldots, x_n \wedge y_n) = f(0,\ldots,0). \qquad \square$$

The next lemma is an application of the 'minimum weight lemma' for the identity gadget.

LEMMA 6.14. *Suppose that* f *has a parity certificate at some point* x *of size at most* k. *Let* K *be a parameter. Suppose that the certificate complexity of* f *at* x *is greater than* $k \cdot K$. *Then we can find a partial assignment* σ *to at most* $k \cdot K$ *variables of* f, *consistent with* x, *such that:*

- $f|_\sigma$ *has a parity certificate at* $x|_\sigma$ *of size at most* k.
- *The minimum weight of this certificate is at least* K.

PROOF. Apply Lemma 5.3 for the function f and the identity function as a gadget g. $\square$

For the last ingredient needed for the proof of the classification theorem, the following lemma shows that parity certificates of high minimum weight always intersect with (regular) certificates of sufficiently small size.

Lemma 6.15. *Suppose that for some function f and some input x the following holds: f has a parity certificate $C_\oplus(x)$ of size k at x whose minimum weight is at least K. Then, for any point y with certificate complexity less than K, there is a non-empty intersection of the corresponding certificates $C(y)$ and $C_\oplus(x)$.*

Proof. Apply Lemma 5.4 with $C_\oplus = C_\oplus(x)$ and $C = C(y)$. □

Now we have everything we need to prove Theorem 6.9.

Proof (Theorem 6.9). Assume that $\mathrm{AND}, \mathrm{XOR} \le g$. The proof in the case of $\mathrm{OR}, \mathrm{XOR} \le g$ is similar. Consider some function f. There is a value S such that $\mathrm{bs}(f) = 9S^5$. If $\mathrm{C}_\oplus(f) \ge S$, then by Corollary 6.11,

$$\mathrm{D}(f \diamond g) \ge \mathrm{D}(f \diamond \mathrm{XOR}) \ge \mathrm{C}_\oplus(f)^{1/12} \ge S^{1/12} \ge \Omega\big(\mathrm{bs}(f)^{1/(12\cdot 5)}\big),$$

and so the theorem holds for $k = 1/60$. So, we can assume that $\mathrm{C}_\oplus(f) < S$. Also, we can assume that for any input, the size of every minimal sensitive block is at most S, otherwise by Lemma 6.12 we get

$$\mathrm{D}(f \diamond g) \ge \mathrm{D}(f \diamond \mathrm{XOR}) = \Omega(S) = \Omega(\mathrm{bs}(f)^{1/5}).$$

Consider any input x with maximal block sensitivity and the corresponding set of minimal sensitive blocks. The certificate complexity of x is at least $9S^5$, since the certificate complexity is at least the block sensitivity. By Lemma 6.14 (for $k = S$ and $K = 2S^3$), we can find a substitution σ such that $f|_\sigma$ has a parity certificate at $x|_\sigma$ of size at most S with minimal weight at least $2S^3$. In addition, the block sensitivity of $f|_\sigma$ at $x|_\sigma$ is at least $9S^5 - 2S^4 \ge 7S^5$ (since we set at most $2S^4$ variables with σ), and the blocks still have size at most S. We fix the substitution σ and consider the restricted function. In what follows, we denote $\hat{f} := f|_\sigma$ and $\hat{x} := x|_\sigma$.

Consider any input y obtained from $\hat{x}$ by flipping one of the minimal sensitive blocks. Suppose that the certificate complexity of this input is at most $2S^3 - S$. Then by Lemma 6.15 (for $k = \mathrm{C}_\oplus(\hat{f}, \hat{x}) \le S$ and $K = 2S^3$), the certificate for y and the parity certificate for $\hat{x}$ intersect. Since $\hat{f}(\hat{x}) \ne \hat{f}(y)$, this leads

to a contradiction. So, the certificate complexity of y should be greater than $2S^3 - S$.

Again, by using Lemma 6.14 (for $k = S$ and $K = S^2 - 1$), there is a substitution τ of at most S^3 variables, such that $y|_\tau$ has a parity certificate of size at most S with minimal weight at least $S^2 - 1$. In addition, the certificate complexity of $y|_\tau$ is at least $2S^3 - S^3 = S^3$.

Since the substitution τ consists of at most S^3 variables, the minimal weight of the parity certificate $\mathrm{C}_\oplus(\hat{f}, \hat{x})|_\tau$ (we apply τ to every linear form in the certificate) is at least $2S^3 - S^3 \geq S^3$. Note that substitution τ can be inconsistent with $\hat{x}$, but we still can make this substitution inside $\mathrm{C}_\oplus(\hat{f}, \hat{x})$ and get a solvable certificate for $\hat{f}|_\tau$. Thus, if we find any point z, consistent with the substitution τ, such that the certificate complexity of $z|_\tau$ is at most $S^2 - S - 1$, then this certificate $\mathrm{C}(\hat{f}|_\tau, z|_\tau)$ should intersect with both $\mathrm{C}_\oplus(\hat{f}, \hat{x})|_\tau$ and $\mathrm{C}_\oplus(\hat{f}, y)|_\tau$ by Lemma 6.15 (for $k = \mathrm{C}_\oplus(\hat{x}) \leq S$ and $K = S^2 - 1$). Since all the these certificates intersect, we have $\hat{f}(\hat{x}) = \hat{f}(z) = \hat{f}(y)$, but this contradicts $\hat{f}(\hat{x}) \neq \hat{f}(y)$.

Now, consider an input z consistent with τ such that all the $*$-variables of τ are set to zeroes. We have shown above that the certificate complexity of $\hat{f}|_\tau$ at $z|_\tau$ is at least $S^2 - S$. Recall that $z|_\tau$ has a block sensitivity witness with all the blocks of size at most S. Since all the blocks together constitute a certificate, we conclude that the block sensitivity of $\hat{f}|_\tau$ at $z|_\tau$ is at least $S - 1$. Since $z|_\tau$ is the all zeroes, by Lemma 6.13 we have

$$\mathrm{D}(f \diamond g) \geq \mathrm{D}(f \diamond \mathrm{AND}) = \Omega(S) = \Omega(\mathrm{bs}(f)^{1/5}). \qquad \square$$

7. Open problems

Matching lower and upper bounds for the certificate complexity lifting Theorem 5.5 shows that one there is a function f such that $\mathrm{C}(f) \geq \Omega(\mathrm{C}_\oplus(f \diamond \mathrm{OR})^2)$. However, Theorem 5.7 shows only that $\mathrm{C}(f) \leq O(\mathrm{C}_\oplus(f \diamond \mathrm{OR})^3)$. Can we show that $\mathrm{C}(f) \leq O(\mathrm{C}_\oplus(f \diamond \mathrm{OR})^2)$?

Generalization of current results to relations Almost all the results discussed above were proved for Boolean functions.

However, the question of proving lifting dichotomies for relations is still open.

One motivation for studying relations instead of functions is the following: any tree-like Resolution refutation of a CNF formula corresponds to a *decision tree*, solving the *falsified clause problem* for this CNF (which is usually a relation rather than a Boolean function). Similarly, any tree-like Res($\oplus$) refutation of some CNF formula corresponds to a *parity decision tree*, solving the falsified clause problem for this CNF. So, any lifting theorem from decision trees to parity decision trees with constant-size gadgets that holds for relations should give us a new way of proving tree-like Res($\oplus$) lower bounds.

Conjunction DAG to parity conjunction DAG lifting Similar to a conjunction DAG, we define *a parity conjunction DAG* with the following two differences:

- Queries are linear forms rather than variables.
- Nodes are annotated by affine subspaces rather than partial assignments. The label $\rho(v_b)$ of a child node v_b, that is attached to the parent node v via an edge labeled b, is a subspace of $\rho(v) \cup \{\ell \leftarrow b\}$, where ℓ is the query in v.

Another possible direction of research is to prove a lifting theorem from the conjunction DAG size to the parity conjunction DAG size. The motivation for this kind of lifting also comes from the proof complexity. Dag-like Resolution refutation can be viewed as a conjunction DAG, and dag-like Res($\oplus$) refutation can be viewed as a parity conjunction DAG. So, proving such lifting theorems for the relations with small enough gadgets can be used to prove lower bounds for Res($\oplus$) refutations, which is a long-standing open problem in proof complexity.

A. Decision tree depth to parity decision size for stifling gadgets

In this section, we paraphrase the proof of Theorem 4 in Chattopadhyay *et al.* (2023). Given a parity decision tree for $f \diamond g$ of size (number of leaves) S, where g is k-stifling for $k \geq 1$, we construct a decision tree for f of depth $(\log S)/k$.

Suppose that f has n inputs and that g has m inputs. We denote the input to f by $x_1, \ldots, x_n$ and the input to $f \diamond g$ by $z_{i,j}$, where $i \in [n]$ and $j \in [m]$. The function $f \diamond g$ is computed by first computing $x_i = g(z_{i,1}, \ldots, z_{i,m})$ and then computing $f(x_1, \ldots, x_n)$.

We will maintain a sequence of *equations* of the form

$$z_{\hat{i},\hat{j}} = z_{i_1,j_1} \oplus \cdots \oplus z_{i_\ell,j_\ell} \oplus b.$$

Initially, the sequence is empty. If $z_{\hat{i},\hat{j}}$ is the left-hand side of one of the equations (in this case, we say that $z_{\hat{i},\hat{j}}$ is *highlighted*), then subsequent equations will not involve $z_{\hat{i},\hat{j}}$.

Given an input x to f, we traverse the parity decision tree for $f \diamond g$ from the root down to the leaves. When we reach a leaf, we output the label listed there. When at an internal vertex v labeled ℓ (a linear combination of $z_{i,j}$'s), we proceed as follows. First, we substitute all the highlighted variables with the corresponding right-hand sides and simplify the result. Let the final result be ℓ'. If ℓ' is constant, then we descend to the corresponding child of v. Otherwise, we descend to the child of v whose subtree has fewer leaves. Suppose that this corresponds to $\ell' = b$. We choose an arbitrary $z_{\hat{i},\hat{j}}$ occurring in ℓ', say $\ell' = z_{\hat{i},\hat{j}} \oplus \ell''$, and add the equation $z_{\hat{i},\hat{j}} = \ell'' \oplus b$. We say that $z_{\hat{i},\hat{j}}$ is highlighted *non-trivially*.

If $z_{\hat{i},\hat{j}}$ is the k'th highlighted input among $z_{\hat{i},1}, \ldots, z_{\hat{i},m}$, then we query $x_{\hat{i}}$, set the non-highlighted $z_{\hat{i},j}$'s so that $g(z_{\hat{i},1}, \ldots, z_{\hat{i},m}) = x_{\hat{i}}$ no matter the value of the highlighted inputs, and add the equations $z_{\hat{i},\hat{j}'} = b_{\hat{i},\hat{j}'}$ corresponding to the $m - k$ inputs set in this way (where $b_{\hat{i},\hat{j}'}$ is the value to which $z_{\hat{i},\hat{j}'}$ is set).

By construction, each time a variable is highlighted non-trivially, the number of leaves decreases by a factor of at least 2. If we query D inputs out of $x_1, \ldots, x_n$, then we have highlighted non-trivially at least kD inputs out of $z_{i,j}$, and so $S \geq 2^{kD}$. Consequently, $D \leq (\log S)/k$.

It remains to show that the output of the decision tree is correct. To see this, it suffices to show that there is some input z such that $g(z) = x$ and z reaches the same leaf of the parity decision tree. Start by querying all x_i not queried so far, and setting the non-highlighted $z_{i,j}$ so that $g(z_{i,1}, \ldots, z_{i,m}) = x_i$, which is possible since at most $k - 1$ of these are highlighted. The system of equations now involves all $z_{i,j}$. Since the system is triangular by construction, it has a solution. The solution corresponds to an input z which reaches the given leaf, and it satisfies $g(z) = x$ by construction.

Acknowledgements

This project received funding from the European Union's Horizon 2020 research and innovation program under grant agreement No 802020-ERC-HARMONIC.

Alexander V. Smal received funding from the European Unions Horizon 2020 research and innovation program under grant agreement No 852870-ERC-SUBMODULAR.

An extended abstract of this paper has appeared as Alekseev *et al.* (2024).

Funding Open access funding provided by Technion - Israel Institute of Technology.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

Publisher's Note
Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

References

Michael Alekhnovich, Jan Johannsen, Toniann Pitassi & Alasdair Urquhart (2007). An exponential separation between regular and general resolution. *Theory Comput.* **3**, 81–102. URL https://doi.org/10.4086/toc.2007.v003a005.

Yaroslav Alekseev, Yuval Filmus & Alexander Smal (2024). Lifting Dichotomies. In *39th Computational Complexity Conference, CCC 2024, July 22-25, 2024, Ann Arbor, MI, USA*, Rahul Santhanam, editor, volume 300 of *LIPIcs*, 9:1–9:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik. URL https://doi.org/10.4230/LIPIcs.CCC.2024.9.

Robert Beals, Harry Buhrman, Richard Cleve, Michele Mosca & Ronald de Wolf (2001). Quantum Lower Bounds by Polynomials. *J. ACM* **48**(4), 778–797. ISSN 0004-5411. URL https://doi.org/10.1145/502090.502097.

Paul Beame & Sajin Koroth (2023). On Disperser/Lifting Properties of the Index and Inner-Product Functions. In *14th Innovations in Theoretical Computer Science Conference, ITCS 2023, January 10-13, 2023, MIT, Cambridge, Massachusetts, USA*, Yael Tauman Kalai, editor, volume 251 of *LIPIcs*, 14:1–14:17. Schloss Dagstuhl - Leibniz-Zentrum für Informatik. URL https://doi.org/10.4230/LIPIcs.ITCS.2023.14.

Eli Ben-Sasson & Avi Wigderson (2001). Short Proofs Are Narrow—Resolution Made Simple. *J. ACM* **48**(2), 149–169. ISSN 0004-5411. URL https://doi.org/10.1145/375827.375835.

A. Bernasconi & B. Codenotti (1999). Spectral analysis of Boolean functions as a graph eigenvalue problem. *IEEE Transactions on Computers* **48**(3), 345–351.

Harry Buhrman & Ronald de Wolf (2002). Complexity measures and decision tree complexity: a survey. *Theoretical Computer Science* **288**(1), 21–43. ISSN 0304-3975. URL https://www.sciencedirect.

com/science/article/pii/S030439750100144X. Complexity and Logic.

Arkadev Chattopadhyay, Michal Koucký, Bruno Loff & Sagnik Mukhopadhyay (2019). Simulation Theorems via Pseudo-Random Properties. *Comput. Complex.* **28**(4), 617–659. ISSN 1016-3328. URL https://doi.org/10.1007/s00037-019-00190-7.

Arkadev Chattopadhyay, Nikhil S. Mande, Swagato Sanyal & Suhail Sherif (2023). Lifting to Parity Decision Trees via Stifling. In *14th Innovations in Theoretical Computer Science Conference (ITCS 2023)*, Yael Tauman Kalai, editor, volume 251 of *Leibniz International Proceedings in Informatics (LIPIcs)*, 33:1–33:20. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany. ISBN 978-3-95977-263-1. ISSN 1868-8969. URL https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITCS.2023.33.

Hubie Chen (2009). A Rendezvous of Logic, Complexity, and Algebra. *ACM Comput. Surv.* **42**(1). ISSN 0360-0300. URL https://doi.org/10.1145/1592451.1592453.

Ankit Garg, Mika Göös, Pritish Kamath & Dmitry Sokolov (2020). Monotone circuit lower bounds from Resolution. *Theory Comput.* **16**, Paper No. 13, 30. URL https://doi.org/10.4086/toc.2020.v016a013.

Mika Göös, Shachar Lovett, Raghu Meka, Thomas Watson & David Zuckerman (2016). Rectangles Are Nonnegative Juntas. *SIAM Journal on Computing* **45**(5), 1835–1869. URL https://doi.org/10.1137/15M103145X.

Mika Göös & Toniann Pitassi (2018). Communication lower bounds via critical block sensitivity. *SIAM J. Comput.* **47**(5), 1778–1806. ISSN 0097-5397. URL https://doi.org/10.1137/16M1082007.

Mika Göös, Toniann Pitassi & Thomas Watson (2018). Deterministic Communication vs. Partition Number. *SIAM Journal on Computing* **47**(6), 2435–2450. URL https://doi.org/10.1137/16M1059369.

Mika Göös, Toniann Pitassi & Thomas Watson (2017). Query-to-Communication Lifting for BPP. In *2017 IEEE 58th Annual Symposium on Foundations of Computer Science (FOCS)*, 132–143.

Hamed Hatami, Kaave Hosseini & Shachar Lovett (2018). Structure of protocols for XOR functions. *SIAM J. Comput.* **47**(1), 208–217. ISSN 0097-5397. URL https://doi.org/10.1137/17M1136869.

Trinh Huynh & Jakob Nordström (2012). On the virtue of succinct proofs: amplifying communication complexity hardness to time-space trade-offs in proof complexity [extended abstract]. In *STOC'12—Proceedings of the 2012 ACM Symposium on Theory of Computing*, 233–247. ACM, New York. URL https://doi.org/10.1145/2213977.2214000.

Alexander Knop, Shachar Lovett, Sam McGuire & Weiqiang Yuan (2021). Log-Rank and Lifting for AND-Functions. In *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2021, 197–208. Association for Computing Machinery, New York, NY, USA. ISBN 9781450380539. URL https://doi.org/10.1145/3406325.3450999.

Eyal Kushilevitz & Noam Nisan (1996). *Communication Complexity*. Cambridge University Press.

Shachar Lovett (2016). Communication is Bounded by Root of Rank. *J. ACM* **63**(1). ISSN 0004-5411. URL https://doi.org/10.1145/2724704.

Shachar Lovett, Raghu Meka, Ian Mertz, Toniann Pitassi & Jiapeng Zhang (2022). Lifting with Sunflowers. In *13th Innovations in Theoretical Computer Science Conference (ITCS 2022)*, Mark Braverman, editor, volume 215 of *Leibniz International Proceedings in Informatics (LIPIcs)*, 104:1–104:24. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany. ISBN 978-3-95977-217-4. ISSN 1868-8969. URL https://drops.dagstuhl.de/opus/volltexte/2022/15700.

László Lovász & Michael Saks (1988). Lattices, mobius functions and communications complexity. In *[Proceedings 1988] 29th Annual Symposium on Foundations of Computer Science*, 81–90.

László Lovász & Michael Saks (1993). Communication complexity and combinatorial lattice theory. *Journal of Computer and System Sciences* **47**(2), 322–349. ISSN 0022-0000. URL https://www.sciencedirect.com/science/article/pii/002200009390035U.

Ran Raz & Pierre Mckenzie (1999). Separation of the Monotone NC Hierarchy. *Combinatorica* **19**.

A. A. Razborov (1992). On the distributional complexity of disjointness. *Theoret. Comput. Sci.* **106**(2), 385–390. ISSN 0304-3975,1879-2294. URL https://doi.org/10.1016/0304-3975(92)90260-M.

Susanna de Rezende, Or Meir, Jakob Nordstrom, Toniann Pitassi, Robert Robere & Marc Vinyals (2020). Lifting with Simple Gadgets and Applications to Circuit and Proof Complexity. In *2020 IEEE 61st Annual Symposium on Foundations of Computer Science (FOCS)*, 24–30.

Alexander A. Sherstov (2010). On quantum-classical equivalence for composed communication problems. *Quantum Inf. Comput.* **10**(5-6), 435–455. ISSN 1533-7146.

Alasdair Urquhart (2011). The Depth of Resolution Proofs. *Studia Logica: An International Journal for Symbolic Logic* **99**(1/3), 349–364. ISSN 00393215, 15728730. URL http://www.jstor.org/stable/41475208.

Shengyu Zhang (2009). On the Tightness of the Buhrman–Cleve–Wigderson Simulation. In *Proceedings of the 20th International Symposium on Algorithms and Computation*, ISAAC '09, 434–440. Springer-Verlag, Berlin, Heidelberg. ISBN 9783642106309. URL https://doi.org/10.1007/978-3-642-10631-6_45.

Zhiqiang Zhang & Yaoyun Shi (2010). On the parity complexity measures of Boolean functions. *Theoretical Computer Science* **411**(26), 2612–2618. ISSN 0304-3975. URL https://www.sciencedirect.com/science/article/pii/S0304397510001647.

Manuscript received 27 June 2024

Yaroslav Alekseev
The Taub Faculty of Computer Science
Technion — Israel Institute of Technology
Haifa, Israel
iaroslava@campus.technion.ac.il

Yuval Filmus
The Taub Faculty of Computer Science and Faculty of Mathematics
Technion — Israel Institute of Technology
Haifa, Israel
yuvalfi@cs.technion.ac.il

Alexander V. Smal
JetBrains Research
Limassol, Cyprus
avsmal@gmail.com